

Chapter 13

SQL for Real Analysis

CTEs, views, and window functions, then ten patterns analysts use every week: top N, Pareto, deduplication, funnels, cohorts, streaks and more.

Draft chapter

Contents

Chapter 13. SQL for Real Analysis

Why this matters

In plain English

13.1 Two practice databases

13.2 Common table expressions: queries in named steps

13.3 Window functions: calculations that keep every row

13.4 Ranking: ROW_NUMBER, RANK, and DENSE_RANK

13.5 LAG and LEAD: comparing a row with its neighbors

13.6 Time windows: growth, targets, and moving averages

13.7 The analyst's pattern library

13.8 Running this chapter in MySQL

Common mistakes and how to spot them

In the real world: Riverstone's year-end business review

Tools

The project: a monthly business review pack

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 13. SQL for Real Analysis

Part II — The Analyst

Chapter at a glance

You will learn to: break a hard question into named steps with common table expressions (CTEs) · save a business definition once as a view · use window functions to calculate shares, rankings, previous values, running totals, and moving averages without losing any rows · apply a library of patterns analysts use every week: top-N per group, deduplication, month-over-month growth, filling missing months, Pareto (ABC) analysis, funnels, cohort retention, streaks, pivots, and data-quality checks.

Before you start: Chapter 12. You should be comfortable with joins, `GROUP BY`, subqueries, and the fan-out trap.

Time needed: 15–20 hours of reading and practice, spread over three weeks.

Tools: PostgreSQL and DBeaver, as in Chapter 12. MySQL works too: section 13.8 shows what changes, and the companion files have a tested MySQL version of every query.

Practice data: two Riverstone databases (section 13.1). Every query was run, and every result shown is the real output.

Why this matters

Chapter 12 taught you to ask a database questions. This chapter teaches you to answer the questions managers actually ask, which are rarely a single `GROUP BY`:

- "How did this month compare with last month?"
- "Who are our top three customers in each segment?"
- "Are we on track for the annual target?"
- "Which customers used to order every month and have gone quiet?"
- "Of the leads that came in, how many became customers?"

In a spreadsheet you'd answer these with helper columns, formulas dragged down thousands of rows, and a lot of care. In SQL, two tools do most of the work. **Common table expressions** let you write a long analysis as a sequence of small, named, readable steps. **Window functions** let each row see its neighbors: the previous month, the customer's total, its rank in a group. Together they turn questions that once took an afternoon into a query you can read in a minute and re-run forever.

These are also the tools that separate a junior SQL user from an analyst in interviews. Almost every data analyst SQL round includes at least one window function question.

In plain English

A CTE is a recipe card with named steps. A recipe doesn't say "cook the thing you made from the thing you chopped". It says: *Step 1, make the dough. Step 2, make the filling. Step 3, fill the dough.* Each step has a name, and later steps refer to earlier ones by name. A CTE lets you write a query the same way: *first total each order, then rank the totals, then keep the top three.*

A window function is looking out of a train window. You stay in your own seat (your row), but you can see the rows around you: the one before, the one after, or the whole carriage. You can say "I'm the third seat from the front" or "this carriage holds 40 people" without anyone leaving their seat. `GROUP BY` is different: it's like counting people by carriage and sending everyone home, leaving one number per carriage.

Keep both pictures in mind. Every technique in this chapter is one of them, or both together.

13.1 Two practice databases

This chapter uses two versions of Riverstone's data, and each section says which one it uses.

The mini database (`riverstone`) is the one from Chapter 12: eight customers and twelve orders in the first quarter of 2026, plus invoices and payments. It's small enough to check every number by eye, so sections 13.2 to 13.5 use it to explain *how* each tool works.

The one-year database (`riverstone_2025`) holds all of 2025: 24 customers, 175 orders, monthly sales targets, and CRM leads. Patterns like growth, seasonality, and retention only show up over time, so sections 13.6 and 13.7 use it. It has the same tables as the mini database (without invoices and payments), plus three more:

Table	One row is	Used for
<code>sales_targets</code>	one month's revenue target	tracking progress against target
<code>leads</code>	one enquiry received (including accidental duplicates)	deduplication, funnels
<code>lead_stage_history</code>	one lead reaching one stage (New, Contacted, Quoted, Won, Lost)	funnel conversion

To set it up, create a database called `riverstone_2025` and run `riverstone_2025_setup.sql` from the companion files (Appendix E), exactly as you did in section 12.3. Check it: `SELECT COUNT(*) FROM orders;` should return `175`. (Using MySQL? Run `riverstone_2025_setup_mysql.sql` instead; it creates the database for you. Section 13.8 covers the rest.)

Tool note. In DBeaver each database is a separate connection. Make sure the editor you're typing in is connected to the right one; the name appears in the editor's toolbar. Running a 2025 query against the mini database is the most common "why doesn't this work?" moment in this chapter.

13.2 Common table expressions: queries in named steps

The problem CTEs solve

Here is the answer to exercise 13 from Chapter 12, *which customers' revenue is above the average customer's revenue?*, written with nested subqueries. It worked, but you had to read it inside-out, and the customer-revenue logic appeared twice. Now the same question as a CTE (mini database):

```
WITH customer_revenue AS (  
    SELECT c.customer_name,  
           SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)) AS revenue  
    FROM customers AS c  
    JOIN orders AS o ON c.customer_id = o.customer_id  
    JOIN order_items AS oi ON o.order_id = oi.order_id  
    WHERE o.status <> 'Cancelled'  
    GROUP BY c.customer_name  
)  
SELECT customer_name, ROUND(revenue, 0) AS revenue  
FROM customer_revenue  
WHERE revenue > (SELECT AVG(revenue) FROM customer_revenue)  
ORDER BY revenue DESC;
```

customer_name	revenue
Coastal Foods	105885
Northgate Distributors	76560

(2 rows)

How it works:

- **WITH customer_revenue AS (...)** defines a step called `customer_revenue`. The query inside the parentheses is an ordinary `SELECT`. Nothing is saved to the database; the name exists only while this statement runs.
- **The final SELECT** reads from `customer_revenue` exactly as if it were a table.
- The average is calculated from the **same named step**, so the revenue logic is written once. If the definition of revenue changes, you change it in one place.

Read it top to bottom as a story: *first work out each customer's revenue; then keep the customers above the average*. That readability is the whole point.

Chaining several steps

A `WITH` clause can define as many steps as you need, separated by commas. Each step can use any step defined above it. Here is the receivables ageing report from section 12.15, rebuilt as a four-step pipeline:

```

WITH payments_per_invoice AS (
    SELECT invoice_id, SUM(amount) AS paid
    FROM payments
    GROUP BY invoice_id
),
invoice_balances AS (
    SELECT i.invoice_id,
           i.order_id,
           i.due_date,
           i.amount - COALESCE(p.paid, 0) AS balance,
           DATE '2026-03-31' - i.due_date AS days_past_due
    FROM invoices AS i
    LEFT JOIN payments_per_invoice AS p ON i.invoice_id = p.invoice_id
),
open_invoices AS (
    SELECT b.*, c.customer_name
    FROM invoice_balances AS b
    JOIN orders AS o ON b.order_id = o.order_id
    JOIN customers AS c ON o.customer_id = c.customer_id
    WHERE b.balance > 0
)
SELECT customer_name,
       SUM(balance) AS total_due,
       SUM(CASE WHEN days_past_due <= 0 THEN balance ELSE 0.00 END) AS not_yet_due,
       SUM(CASE WHEN days_past_due BETWEEN 1 AND 30 THEN balance ELSE 0.00 END) AS overdue_1_30,
       SUM(CASE WHEN days_past_due BETWEEN 31 AND 60 THEN balance ELSE 0.00 END) AS overdue_31_60,
       SUM(CASE WHEN days_past_due > 60 THEN balance ELSE 0.00 END) AS overdue_60_plus
FROM open_invoices
GROUP BY customer_name
ORDER BY total_due DESC;

```

customer_name	total_due	not_yet_due	overdue_1_30	overdue_31_60	overdue_60_plus
Northgate Distributors	46560.00	0.00	46560.00	0.00	0.00
Sunrise Caterers	23325.00	0.00	23325.00	0.00	0.00
Coastal Foods	12625.00	0.00	12625.00	0.00	0.00
Sharma Hardware	11700.00	11700.00	0.00	0.00	0.00
Patel Kitchenware	6250.00	0.00	0.00	6250.00	0.00
(5 rows)					

The result is identical to Chapter 12's, but the query now reads like the method you'd explain to a colleague:

1. **payments_per_invoice** : add up payments per invoice (this is the fan-out fix, now with a name).
2. **invoice_balances** : work out what's still owed on each invoice, and how many days past due it is.
3. **open_invoices** : keep invoices with a balance, and attach the customer's name.
4. **Final SELECT** : spread balances into ageing columns.

Calculating `days_past_due` once in step 2 also made the `CASE` conditions in step 4 much simpler than in Chapter 12.

Real-life habit: debug one step at a time. When a CTE query gives a strange answer, replace the final `SELECT` with `SELECT * FROM invoice_balances;` (or whichever step you suspect) and look at the intermediate rows. Because every step has a name, you can inspect any stage of the pipeline without rewriting anything. Experienced analysts build long CTE queries exactly this way: write one step, check it, add the next.

CTE, subquery, view, or temporary table?

All four let you reuse the result of a query. The difference is how long the result lives and who can use it:

Tool	Lives for	Best for
Subquery	this one spot in the query	a small, one-off calculation, like a single average
CTE	this one statement	breaking one analysis into readable steps
View	permanently, until dropped	a definition many queries and people should share
Temporary table	until you disconnect	a heavy intermediate result you'll query many times in one session

Views: define it once, use it everywhere

In Chapter 12 you typed the net revenue formula, `quantity * unit_price * (1 - discount_pct / 100)`, more than a dozen times, along with the rule "exclude cancelled orders". Every copy was a chance for a typo, and if finance ever changes the rule, every query must be found and changed.

A **view** is a saved query with a name. It stores no data of its own; each time you query it, the database runs the saved `SELECT`. Create this one in **both** databases, because the rest of the chapter uses it:

```
CREATE VIEW sales_lines AS
SELECT o.order_id,
       o.order_date,
       o.customer_id,
       o.sales_rep_id,
       o.status,
       oi.product_id,
       p.category,
       oi.quantity,
       oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100) AS net_revenue,
       oi.quantity * p.unit_cost AS product_cost
FROM orders AS o
JOIN order_items AS oi ON o.order_id = oi.order_id
JOIN products AS p ON oi.product_id = p.product_id
WHERE o.status <> 'Cancelled';
```

Now revenue by category (mini database) is three lines:

```
SELECT category, ROUND(SUM(net_revenue), 0) AS net_revenue
FROM sales_lines
GROUP BY category
ORDER BY net_revenue DESC;
```

category	net_revenue
Industrial	161385
Storage	81810
Kitchen	80735

(3 rows)

Same numbers as section 12.9, with none of the repetition. Notice two things about the view:

- **Its grain is one order line.** Summing `net_revenue` is safe; counting rows counts lines, not orders. The fan-out lesson still applies, so use `COUNT(DISTINCT order_id)` for orders.
- **It contains a business rule:** cancelled orders are excluded. That's the point of a view: everyone who uses `sales_lines` gets the same definition. It's also a responsibility. Document the rule, and agree it with finance before other people build reports on it.

Real-life example: "the one true revenue". In many companies the sales dashboard, the finance report, and the CEO's slide show three different revenue numbers for the same month, because three people wrote three slightly different queries. A shared view (and, later, the dbt models of Chapter 32) is how data teams end that argument.

13.3 Window functions: calculations that keep every row

The idea

Start with a question from Riverstone's sales head, using the mini database: "For each order this quarter, what share of the quarter's revenue did it bring in?"

To answer it, each order row needs two numbers: its own revenue, and the quarter's total. `GROUP BY` can give you the total, but only by collapsing all the orders into one row:

```
WITH order_totals AS (
  SELECT order_id, customer_id, order_date, SUM(net_revenue) AS order_revenue
  FROM sales_lines
  GROUP BY order_id, customer_id, order_date
)
SELECT ROUND(SUM(order_revenue), 0) AS quarter_total
FROM order_totals;
```

quarter_total
323930

(1 row)

A **window function** calculates the same total but *keeps every row*, attaching the total to each one:

```

WITH order_totals AS (
  SELECT order_id, customer_id, order_date, SUM(net_revenue) AS order_revenue
  FROM sales_lines
  GROUP BY order_id, customer_id, order_date
)
SELECT order_id,
  ROUND(order_revenue, 0) AS order_revenue,
  ROUND(SUM(order_revenue) OVER (), 0) AS quarter_total,
  ROUND(100.0 * order_revenue / SUM(order_revenue) OVER (), 1) AS pct_of_quarter
FROM order_totals
ORDER BY order_id;

```

order_id	order_revenue	quarter_total	pct_of_quarter
5001	14700	323930	4.5
5002	73260	323930	22.6
5003	16250	323930	5.0
5005	14550	323930	4.5
5006	14640	323930	4.5
5007	32625	323930	10.1
5008	23325	323930	7.2
5009	76560	323930	23.6
5010	20100	323930	6.2
5011	11700	323930	3.6
5012	26220	323930	8.1

(11 rows)

The magic word is **OVER**. `SUM(order_revenue)` on its own is an aggregate that collapses rows.

`SUM(order_revenue) OVER ()` is a window function: *add up order_revenue across a set of rows (the window), and show the result on every row*. The empty parentheses mean the window is **all rows** in the result.

Now the answer jumps out: two orders, 5009 (Northgate) and 5002 (Coastal Foods), brought in 46% of the quarter's revenue. That's a concentration risk worth mentioning.

Notice the query also shows the CTE and the view working together: `sales_lines` supplies clean order lines, `order_totals` turns them into one row per order, and the window function adds the share.

PARTITION BY: a window for each group

The next question: "For each customer, how is their revenue split across their orders?" Now each order needs its **customer's** total, not the quarter's. `PARTITION BY` splits the rows into groups, and the window restarts for each group:

```

WITH order_totals AS (
  SELECT order_id, customer_id, SUM(net_revenue) AS order_revenue
  FROM sales_lines
  GROUP BY order_id, customer_id
)
SELECT c.customer_name,
       t.order_id,
       ROUND(t.order_revenue, 0) AS order_revenue,
       ROUND(SUM(t.order_revenue) OVER (PARTITION BY t.customer_id), 0) AS customer_total,
       ROUND(100.0 * t.order_revenue / SUM(t.order_revenue) OVER (PARTITION BY t.customer_id), 1) AS pct_of_customer
FROM order_totals AS t
JOIN customers AS c ON t.customer_id = c.customer_id
ORDER BY c.customer_name, t.order_id;

```

customer_name	order_id	order_revenue	customer_total	pct_of_customer
Coastal Foods	5002	73260	105885	69.2
Coastal Foods	5007	32625	105885	30.8
Green Leaf Hotels	5010	20100	20100	100.0
Metro Mart	5006	14640	40860	35.8
Metro Mart	5012	26220	40860	64.2
Northgate Distributors	5009	76560	76560	100.0
Patel Kitchenware	5003	16250	16250	100.0
Sharma Hardware	5001	14700	40950	35.9
Sharma Hardware	5005	14550	40950	35.5
Sharma Hardware	5011	11700	40950	28.6
Sunrise Caterers	5008	23325	23325	100.0

(11 rows)

Read the Sharma Hardware rows: three orders, each shown with the customer's total of ₹40,950 and its own share. Sharma orders steadily; Coastal Foods placed one very large order and a smaller one. Figure 13.1 shows the difference between this and `GROUP BY`.

order_totals (5 rows)

customer	order_id	order_revenue
Metro Mart	5006	14640
Metro Mart	5012	26220
Sharma Hardware	5001	14700
Sharma Hardware	5005	14550
Sharma Hardware	5011	11700

Same input, two different questions:

GROUP BY answers: what is each customer's total?
 A window answers: what is each order, and
 how big is it compared with its customer's total?

Windows add information without losing rows.

GROUP BY customer

2 rows: the orders are collapsed

customer	total
Metro Mart	40860
Sharma Hardware	40950

SUM(order_revenue) ... GROUP BY customer

SUM(...) OVER (PARTITION BY customer)

5 rows: every order kept, total added alongside

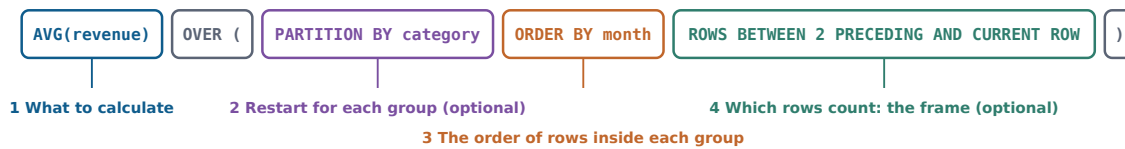
customer	order_id	order_revenue	customer_total
Metro Mart	5006	14640	40860
Metro Mart	5012	26220	40860
Sharma Hardware	5001	14700	40950
Sharma Hardware	5005	14550	40950
Sharma Hardware	5011	11700	40950

Figure 13.1 — `GROUP BY` returns one row per customer. `PARTITION BY` returns every order, with the customer's total added alongside.

Spreadsheet link. In Excel you'd get the customer total with `SUMIFS` in a helper column, one formula per row. `SUM(...) OVER (PARTITION BY ...)` is that helper column, calculated for every row at once.

The anatomy of a window

Every window function has the same shape. Only the function itself is required; the three parts inside `OVER ( )` are optional, and you add each one when the question needs it.



The frame slides down the rows. For April, the 3-month window is February, March and April:

Jan 2025	202,640	
Feb 2025	253,664	← 2 PRECEDING
Mar 2025	278,008	
Apr 2025	210,282	← CURRENT ROW
May 2025	329,359	
Jun 2025	186,928	

moving_avg_3m for April

$(253,664 + 278,008 + 210,282) \div 3 = 247,318$

In May, the frame moves on: March, April, May.

Figure 13.2 – The four parts of a window function. The frame example uses Riverstone's 2025 monthly revenue from section 13.6.

Part	Question it answers	Example
The function	What should I calculate?	<code>SUM</code> , <code>AVG</code> , <code>COUNT</code> , <code>ROW_NUMBER</code> , <code>RANK</code> , <code>LAG</code>
PARTITION BY	Should the calculation restart for each group?	<code>PARTITION BY customer_id</code>
ORDER BY	In what order should rows be lined up inside each group?	<code>ORDER BY order_date</code>
Frame (<code>ROWS BETWEEN ...</code>)	Which of those rows count for this row?	<code>ROWS BETWEEN 2 PRECEDING AND CURRENT ROW</code>

Where windows run, and why you can't filter on them directly

Window functions are calculated **after** `WHERE` , `GROUP BY` , and `HAVING` , and **before** `ORDER BY` and `LIMIT` . In the execution order from section 12.11, they belong with `SELECT` , at step 5.

That has two consequences you'll use constantly:

1. **Window functions can use aggregates.** Because grouping has already happened, you can write `SUM(SUM(net_revenue)) OVER ( )` : the inner `SUM` belongs to `GROUP BY` , the outer one is the window.
2. **You can't put a window function in `WHERE` .** `WHERE` runs at step 2, before windows exist:

```
SELECT order_id, customer_id, order_date
FROM orders
WHERE ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) = 1;
```

ERROR: window functions are not allowed in WHERE

The fix is the pattern you'll see in almost every query from here on: **calculate the window in a CTE, then filter in the next step.**

Dialect note: QUALIFY. Snowflake, BigQuery, Databricks, and DuckDB support a `QUALIFY` clause that filters on window functions directly: `QUALIFY ROW_NUMBER() OVER (...) = 1`. PostgreSQL, MySQL, and SQL Server don't, so this book uses the CTE pattern, which works everywhere.

13.4 Ranking: ROW_NUMBER, RANK, and DENSE_RANK

Three ways to number rows

Ranking sounds simple until two rows tie. In the mini database, the Food Container Set and the Storage Box 10L both sold exactly 85 units. Here are the three ranking functions side by side:

```
WITH units AS (
  SELECT p.product_name, COALESCE(SUM(s.quantity), 0) AS units_sold
  FROM products AS p
  LEFT JOIN sales_lines AS s ON p.product_id = s.product_id
  GROUP BY p.product_name
)
SELECT product_name,
       units_sold,
       ROW_NUMBER() OVER (ORDER BY units_sold DESC, product_name) AS row_num,
       RANK() OVER (ORDER BY units_sold DESC) AS rank_no,
       DENSE_RANK() OVER (ORDER BY units_sold DESC) AS dense_rank_no
FROM units
ORDER BY units_sold DESC, product_name;
```

product_name	units_sold	row_num	rank_no	dense_rank_no
Water Bottle 1L	230	1	1	1
Industrial Crate	125	2	2	2
Food Container Set	85	3	3	3
Storage Box 10L	85	4	3	3
Storage Box 25L	60	5	5	4
Garden Chair	0	6	6	5

(6 rows)

Units sold, ranked three ways

product_name	units	ROW_NUMBER	RANK	DENSE_RANK
Water Bottle 1L	230	1	1	1
Industrial Crate	125	2	2	2
Food Container Set	85	3	3	3
Storage Box 10L	85	4	3	3
Storage Box 25L	60	5	5	4
Garden Chair	0	6	6	5

ROW_NUMBER

Always 1, 2, 3, 4... Ties get different numbers, so add a tie-breaker to ORDER BY (here: product name).

RANK

Ties share a number, then it skips: 3, 3, 5. Like a race: two joint third places, then fifth.

DENSE_RANK

Ties share a number, no gaps: 3, 3, 4. Use it for 'the top 3 distinct sales levels'.

Figure 13.3 – The same tie, numbered three ways.

- **ROW_NUMBER()** gives every row a different number, even on a tie. Which of the tied rows gets 3 and which gets 4 is up to you: that's why `product_name` is added to its `ORDER BY` as a **tie-breaker**. Without one, the database may choose differently tomorrow, and your report changes for no reason.
- **RANK()** gives tied rows the same rank, then **skips**: 3, 3, 5. It's how a merit list or a race works: two students tie for third place, and the next student is fifth, because four students scored higher.
- **DENSE_RANK()** gives tied rows the same rank **without skipping**: 3, 3, 4. Use it when you want "the top three distinct sales levels".

Which one is right is a business question. "Give a bonus to the top 3 reps": if two tie for third, does the company pay four bonuses (`RANK` or `DENSE_RANK` ≤ 3) or pick one (`ROW_NUMBER` ≤ 3 with a stated tie-breaker)? Ask before you write the query.

The latest record per group

A very common request: "Show each customer's most recent order." Chapter 12 solved a version of this with a correlated subquery. `ROW_NUMBER()` is cleaner and handles ties safely:

```
WITH ranked AS (
  SELECT c.customer_name,
         o.order_id,
         o.order_date,
         o.status,
         ROW_NUMBER() OVER (PARTITION BY o.customer_id
                           ORDER BY o.order_date DESC, o.order_id DESC) AS recency_rank
  FROM orders AS o
  JOIN customers AS c ON o.customer_id = c.customer_id
)
SELECT customer_name, order_id, order_date, status
FROM ranked
WHERE recency_rank = 1
ORDER BY order_date DESC;
```

customer_name	order_id	order_date	status
Metro Mart	5012	2026-03-15	Pending
Sharma Hardware	5011	2026-03-10	Shipped
Green Leaf Hotels	5010	2026-03-03	Delivered
Northgate Distributors	5009	2026-02-25	Shipped
Sunrise Caterers	5008	2026-02-19	Delivered
Coastal Foods	5007	2026-02-11	Delivered
Patel Kitchenware	5003	2026-01-14	Delivered

(7 rows)

How it works:

- `PARTITION BY o.customer_id` numbers each customer's orders separately, starting from 1.
- `ORDER BY o.order_date DESC, o.order_id DESC` makes 1 the newest order. The `order_id` tie-breaker decides if a customer places two orders on the same day.
- The CTE calculates the numbers; the final query keeps `recency_rank = 1`.

This **"number, then keep 1"** pattern solves a whole family of questions: each customer's first purchase (`ORDER BY order_date ASC`), each product's highest-value order, each employee's latest address, the current price of each product from a price-history table. You'll use it every week.

13.5 LAG and LEAD: comparing a row with its neighbors

Looking back

`LAG(column)` returns the value of `column` from the **previous** row in the window; `LEAD(column)` returns the value from the **next** row. The question here is a real sales-team concern: *"How many days pass between a customer's orders?"*

```
SELECT c.customer_name,
       o.order_id,
       o.order_date,
       LAG(o.order_date) OVER (PARTITION BY o.customer_id ORDER BY o.order_date) AS
previous_order,
       o.order_date - LAG(o.order_date) OVER (PARTITION BY o.customer_id ORDER BY o.order_date) AS
days_since_previous
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.customer_id
WHERE o.status <> 'Cancelled'
ORDER BY c.customer_name, o.order_date;
```

customer_name	order_id	order_date	previous_order	days_since_previous
Coastal Foods	5002	2026-01-09		
Coastal Foods	5007	2026-02-11	2026-01-09	33
Green Leaf Hotels	5010	2026-03-03		
Metro Mart	5006	2026-02-06		
Metro Mart	5012	2026-03-15	2026-02-06	37
Northgate Distributors	5009	2026-02-25		
Patel Kitchenware	5003	2026-01-14		
Sharma Hardware	5001	2026-01-05		
Sharma Hardware	5005	2026-02-02	2026-01-05	28
Sharma Hardware	5011	2026-03-10	2026-02-02	36
Sunrise Caterers	5008	2026-02-19		

(11 rows)

How it works:

- `PARTITION BY o.customer_id` keeps each customer's orders separate, so Coastal Foods' first order doesn't look back at somebody else's.
- `ORDER BY o.order_date` lines the orders up in time, so "previous" means the previous order *in date order*.
- A customer's first order has no previous row, so `LAG` returns **NULL**, and the subtraction is NULL too. That's correct: there is no gap to measure.
- The `WHERE` removes cancelled orders **before** the window is calculated (step 2 before step 5), so Green Leaf's cancelled January order doesn't count as a purchase.

Real-life example: the reorder rhythm. Sharma Hardware reorders roughly every 28–36 days. If a month and a half passes with no order, something has changed: a competitor, a problem with the last delivery, a new purchasing manager. A sales team that knows each customer's rhythm can call *before* the customer is lost. Section 13.7 turns this into a full "at-risk customers" report.

`LAG` and `LEAD` take two optional extra arguments: how many rows to look back, and a default instead of `NULL`. `LAG(revenue, 12, 0) OVER (ORDER BY month)` means *the value 12 rows back, or 0 if there isn't one*, which is exactly "the same month last year" on a monthly table.

13.6 Time windows: growth, targets, and moving averages

From here on, switch to the **one-year database** (`riverstone_2025`), and make sure you've created the `sales_lines` view in it (section 13.2). First, a quick look at the year:

```
SELECT COUNT(DISTINCT order_id) AS orders,
       COUNT(DISTINCT customer_id) AS customers,
       ROUND(SUM(net_revenue), 0) AS net_revenue
FROM sales_lines;
```

orders	customers	net_revenue
173	23	4335471

(1 row)

173 non-cancelled orders from 23 customers, worth about ₹43 lakh. (One customer, Home Plus, signed up but never ordered.)

Month-over-month growth

"How did each month compare with the month before?" is the most-asked question in business reporting. Build monthly totals in a CTE, then use `LAG`:

```
WITH monthly AS (
  SELECT DATE_TRUNC('month', order_date)::date AS month,
         SUM(net_revenue) AS revenue
  FROM sales_lines
  GROUP BY DATE_TRUNC('month', order_date)
)
SELECT month,
       ROUND(revenue, 0) AS revenue,
       ROUND(LAG(revenue) OVER w, 0) AS previous_month,
       ROUND(revenue - LAG(revenue) OVER w, 0) AS change_amount,
       ROUND(100.0 * (revenue - LAG(revenue) OVER w) / LAG(revenue) OVER w, 1) AS pct_change
FROM monthly
WINDOW w AS (ORDER BY month)
ORDER BY month;
```

month	revenue	previous_month	change_amount	pct_change
2025-01-01	202640			
2025-02-01	253664	202640	51024	25.2
2025-03-01	278008	253664	24344	9.6
2025-04-01	210282	278008	-67726	-24.4
2025-05-01	329359	210282	119077	56.6
2025-06-01	186928	329359	-142431	-43.2
2025-07-01	232692	186928	45764	24.5
2025-08-01	329282	232692	96590	41.5
2025-09-01	558315	329282	229033	69.6
2025-10-01	681071	558315	122756	22.0
2025-11-01	633408	681071	-47663	-7.0
2025-12-01	439824	633408	-193585	-30.6

(12 rows)

Two new ideas:

- **WINDOW w AS (ORDER BY month)** gives a window definition a name, so you can write **OVER w** four times instead of repeating **OVER (ORDER BY month)**. It goes after **FROM / WHERE / GROUP BY** and before **ORDER BY**. (MySQL supports named windows too. SQL Server doesn't before its 2022 version; there you repeat the definition.)
- **Percent change** is $(\text{this month} - \text{last month}) \div \text{last month}$. January has no previous month, so it shows NULL rather than a misleading 0% or an error.

Reading it like an analyst. The pattern is seasonal, not random: revenue dips in June and July (the monsoon months are slow for construction-linked and hospitality buyers), climbs through the festive season to a peak in October, and eases in December. Month-over-month percentages exaggerate that seasonality: a -43% June looks alarming but is mostly the calendar. That's why businesses also compare each month with **the same month last year**, which you'd do with **LAG(revenue, 12)** once you have two years of data.

Running totals: are we on track for the year?

Riverstone sets a revenue target for every month in **sales_targets**. A monthly miss matters less than the question every sales head asks from October onwards: *are we on track for the year?* That needs **year-to-date** (running) totals of both revenue and target:

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date)::date AS month, SUM(net_revenue) AS revenue
    FROM sales_lines
    GROUP BY DATE_TRUNC('month', order_date)
)
SELECT m.month,
       ROUND(m.revenue, 0) AS revenue,
       ROUND(t.target_revenue, 0) AS target,
       ROUND(SUM(m.revenue) OVER w, 0) AS ytd_revenue,
       ROUND(SUM(t.target_revenue) OVER w, 0) AS ytd_target,
       ROUND(100.0 * SUM(m.revenue) OVER w / SUM(t.target_revenue) OVER w, 1) AS ytd_pct_of_target
FROM monthly AS m
JOIN sales_targets AS t ON t.target_month = m.month
WINDOW w AS (ORDER BY m.month)
ORDER BY m.month;
```

month	revenue	target	ytd_revenue	ytd_target	ytd_pct_of_target
2025-01-01	202640	300000	202640	300000	67.5
2025-02-01	253664	300000	456304	600000	76.1
2025-03-01	278008	320000	734312	920000	79.8
2025-04-01	210282	320000	944593	1240000	76.2
2025-05-01	329359	320000	1273952	1560000	81.7
2025-06-01	186928	300000	1460880	1860000	78.5
2025-07-01	232692	280000	1693572	2140000	79.1
2025-08-01	329282	320000	2022854	2460000	82.2
2025-09-01	558315	380000	2581169	2840000	90.9
2025-10-01	681071	520000	3262240	3360000	97.1
2025-11-01	633408	500000	3895648	3860000	100.9
2025-12-01	439824	380000	4335471	4240000	102.3

(12 rows)

How it works: `SUM(m.revenue) OVER (ORDER BY m.month)` adds up revenue from the first month **up to and including** the current month. For March, that's January + February + March = ₹734,312. Change the month and the total grows. That's a running total.

The story in the numbers. Riverstone was behind target for ten straight months, at under 80% of the year-to-date target through July. A monthly report would have shown a string of misses. The running total shows something more useful: September and October closed most of the gap, November crossed the line, and the year finished at **102.3% of target**. If a manager had asked in August "*can we still make it?*", the answer was in the gap: ₹437,146 behind with the strongest season still to come.

Moving averages: seeing the trend through the noise

Monthly revenue jumps around. A **moving average** smooths it by averaging each month with the months just before it:

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date)::date AS month, SUM(net_revenue) AS revenue
    FROM sales_lines
    GROUP BY DATE_TRUNC('month', order_date)
)
SELECT month,
    ROUND(revenue, 0) AS revenue,
    ROUND(AVG(revenue) OVER (ORDER BY month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 0) AS
moving_avg_3m,
    COUNT(*)          OVER (ORDER BY month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)      AS
months_in_window
FROM monthly
ORDER BY month;
```

month	revenue	moving_avg_3m	months_in_window
2025-01-01	202640	202640	1
2025-02-01	253664	228152	2
2025-03-01	278008	244771	3
2025-04-01	210282	247318	3
2025-05-01	329359	272549	3
2025-06-01	186928	242189	3
2025-07-01	232692	249660	3
2025-08-01	329282	249634	3
2025-09-01	558315	373430	3
2025-10-01	681071	522889	3
2025-11-01	633408	624265	3
2025-12-01	439824	584767	3
(12 rows)			

ROWS BETWEEN 2 PRECEDING AND CURRENT ROW is the **frame**: for each month, use this row and the two rows before it (Figure 13.2 shows April's frame). While monthly revenue swings from ₹210,282 to ₹329,359 and down to ₹186,928 between April and June, the moving average stays between about ₹242,000 and ₹273,000: the underlying trend was flat until the festive season lifted it sharply in September.

The `months_in_window` column exposes a detail people miss: January and February don't have two earlier months, so their "3-month" average is really a 1- and 2-month average. In a published chart, either label the first points clearly or start the series in March.

The frame vocabulary is small:

Frame boundary	Means
UNBOUNDED PRECEDING	the first row of the partition
n PRECEDING	n rows before the current row
CURRENT ROW	this row
n FOLLOWING	n rows after the current row
UNBOUNDED FOLLOWING	the last row of the partition

The hidden frame trap: ROWS versus RANGE

When you write `ORDER BY` inside `OVER` but **no frame**, SQL uses a default frame: `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. `RANGE` treats rows with the **same ORDER BY value as one step**. That's invisible until two rows tie, and then running totals jump. Riverstone had three orders on 4 May 2025:

```

WITH early_may AS (
  SELECT order_id, order_date, SUM(net_revenue) AS order_revenue
  FROM sales_lines
  WHERE order_date BETWEEN '2025-05-01' AND '2025-05-06'
  GROUP BY order_id, order_date
)
SELECT order_id,
       order_date,
       ROUND(order_revenue, 0) AS order_revenue,
       ROUND(SUM(order_revenue) OVER (ORDER BY order_date), 0) AS running_default,
       ROUND(SUM(order_revenue) OVER (ORDER BY order_date, order_id
                                     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), 0) AS
running_rows
FROM early_may
ORDER BY order_date, order_id;

```

order_id	order_date	order_revenue	running_default	running_rows
10043	2025-05-04	23650	97735	23650
10044	2025-05-04	51310	97735	74960
10045	2025-05-04	22775	97735	97735
10046	2025-05-06	5805	103540	103540

(4 rows)

With the default, all three 4 May orders show ₹97,735, the total for the whole day, because `RANGE` includes every row that ties on `order_date`. With `ROWS` and a tie-breaker, the total grows one order at a time.

Neither is wrong. For a daily running total, the default is exactly right. For "running total order by order", it's a bug that looks like a data problem. **Rule: whenever you write a running total, write the frame explicitly** so the next reader doesn't have to know the default.

Watch out: LAST_VALUE. The same default frame surprises people with `LAST_VALUE`. With `ORDER BY` and no frame, the window ends at the current row, so `LAST_VALUE(x)` just returns the current row's `x`. To get the true last value in the partition, write `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`, or simply use `FIRST_VALUE` with the order reversed.

13.7 The analyst's pattern library

The rest of the chapter is a set of **patterns**: recurring business questions, each with a reusable query shape. Real analysis work is mostly recognizing which pattern a new question belongs to, then adapting it. Every pattern below uses the one-year database.

Each pattern follows the same format: **the question, the idea, the query, the result, how it works, and where else you'll use it**.

Pattern 1: Top N per group

The question. "Who are the top three customers in each segment, and how much of their segment do they account for?"

The idea. Total revenue per customer (a CTE), rank customers *within* each segment (`RANK() OVER (PARTITION BY segment ...)`), then keep ranks 1 to 3.

```
WITH customer_revenue AS (  
    SELECT c.segment, c.customer_name, SUM(s.net_revenue) AS revenue  
    FROM sales_lines AS s  
    JOIN customers AS c ON s.customer_id = c.customer_id  
    GROUP BY c.segment, c.customer_name  
)  
ranked AS (  
    SELECT segment,  
           customer_name,  
           revenue,  
           RANK() OVER (PARTITION BY segment ORDER BY revenue DESC) AS rank_in_segment,  
           100.0 * revenue / SUM(revenue) OVER (PARTITION BY segment) AS pct_of_segment  
    FROM customer_revenue  
)  
SELECT segment, rank_in_segment, customer_name, ROUND(revenue, 0) AS revenue, ROUND(pct_of_segment,  
1) AS pct_of_segment  
FROM ranked  
WHERE rank_in_segment <= 3  
ORDER BY segment, rank_in_segment;
```

segment	rank_in_segment	customer_name	revenue	pct_of_segment
Hospitality	1	Green Leaf Hotels	331389	29.0
Hospitality	2	Fresh Bowl Kitchens	296876	25.9
Hospitality	3	Spice Route Restaurants	167776	14.7
Retail	1	Sharma Hardware	502775	33.8
Retail	2	Metro Mart	329298	22.1
Retail	3	Evergreen Mart	198278	13.3
Wholesale	1	Harbour Traders	412681	24.2
Wholesale	2	Northgate Distributors	353089	20.7
Wholesale	3	Coastal Foods	323631	19.0
(9 rows)				

How it works. Two windows share one partition: `RANK()` orders customers inside each segment, and `SUM(revenue) OVER (PARTITION BY segment)` gives the segment total for the share. The share is calculated *before* filtering to the top three, which is why it's a share of the **whole** segment. (Filter first and each segment's top three would add up to 100%, a subtle and common mistake.)

Reading it. In Retail, Sharma Hardware alone is a third of the segment. Losing that one account would hurt Retail far more than losing any single Wholesale customer, where revenue is spread more evenly.

Where else you'll use it. Top five products per region, the three longest delays per supplier, each branch's best salesperson, the two most recent visits per patient.

Pattern 2: Pareto and ABC analysis

The question. "Which customers make up 80% of our revenue?"

The idea. The **Pareto principle** (the "80/20 rule") observes that a small share of customers, products, or problems usually accounts for most of the value. **ABC analysis** puts it to work: sort by revenue, calculate a running share, and label the customers that make up the first 80% as **A**, the next 15% as **B**, and the rest as **C**. Warehouses use exactly the same method to decide which stock to count most often.

```

WITH customer_revenue AS (
    SELECT c.customer_name, SUM(s.net_revenue) AS revenue
    FROM sales_lines AS s
    JOIN customers AS c ON s.customer_id = c.customer_id
    GROUP BY c.customer_name
),
cumulative AS (
    SELECT customer_name,
           revenue,
           100.0 * revenue / SUM(revenue) OVER () AS pct_of_total,
           100.0 * SUM(revenue) OVER (ORDER BY revenue DESC, customer_name
                                     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
                                     / SUM(revenue) OVER () AS cumulative_pct
    FROM customer_revenue
)
SELECT customer_name,
       ROUND(revenue, 0) AS revenue,
       ROUND(pct_of_total, 1) AS pct_of_total,
       ROUND(cumulative_pct, 1) AS cumulative_pct,
       CASE WHEN cumulative_pct - pct_of_total < 80 THEN 'A'
            WHEN cumulative_pct - pct_of_total < 95 THEN 'B'
            ELSE 'C' END AS abc_class
FROM cumulative
ORDER BY revenue DESC;

```

customer_name	revenue	pct_of_total	cumulative_pct	abc_class
Sharma Hardware	502775	11.6	11.6	A
Harbour Traders	412681	9.5	21.1	A
Northgate Distributors	353089	8.1	29.3	A
Green Leaf Hotels	331389	7.6	36.9	A
Metro Mart	329298	7.6	44.5	A
Coastal Foods	323631	7.5	52.0	A
Deccan Packaging	299010	6.9	58.9	A
Fresh Bowl Kitchens	296876	6.8	65.7	A
Western Logistics	248728	5.7	71.4	A
Evergreen Mart	198278	4.6	76.0	A
Patel Kitchenware	174148	4.0	80.0	A
Spice Route Restaurants	167776	3.9	83.9	B
Kitchen Kraft	127978	3.0	86.9	B
Sea Breeze Hotel	92928	2.1	89.0	B
Om Sai Provisions	66399	1.5	90.5	B
Prime Wholesale	65522	1.5	92.0	B
Lakeview Resorts	61973	1.4	93.5	B
City Needs Store	55650	1.3	94.8	B
Royal Banquets	54366	1.3	96.0	B
Sunrise Caterers	47674	1.1	97.1	C
Blue Bay Cafe	46208	1.1	98.2	C
Tasty Tiffins	44850	1.0	99.2	C
Festive Gifts Co	34250	0.8	100.0	C

(23 rows)

How it works.

- `SUM(revenue) OVER (ORDER BY revenue DESC ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)` is a running total from the biggest customer down. Dividing by `SUM(revenue) OVER ()` turns it into a running percentage.
- `customer_name` in the window's `ORDER BY`, together with the explicit `ROWS` frame, keeps the running total correct even if two customers had identical revenue (the trap from section 13.6).

- The class uses `cumulative_pct - pct_of_total`: the running share **before** this customer. A customer is class A if the list hadn't reached 80% when they were added. That's why Patel Kitchenware, which takes the total from 76.0% to exactly 80.0%, is still A.

Reading it. Eleven of 23 customers (under half) bring in 80% of revenue. Riverstone's is a flatter distribution than the classic 80/20, which is good news: no single customer can sink the business. The A list is where account managers should spend most of their time; the C list may be served more cheaply, for example through online ordering.

Where else you'll use it. Products (exercise 9), inventory (which items to count weekly), customer complaints (which few causes create most tickets), and costs (which suppliers make up most spending).

Pattern 3: Remove duplicates, keep one

The question. "The CRM has duplicate leads. How many real enquiries did we get?"

The idea. The same enquiry submitted twice shares an email address, even if the company name was typed differently. Number the rows within each email address, oldest first, and keep row 1.

A real-life cause: someone fills in the website form, the page is slow, they click Submit again, and then they fill it in once more in capital letters to be safe. Your lead count is now three times too high. First, look at the duplicates:

```
WITH numbered AS (  
    SELECT lead_id,  
           company_name,  
           email,  
           created_at,  
           ROW_NUMBER() OVER (PARTITION BY LOWER(email) ORDER BY created_at, lead_id) AS  
submission_no,  
           COUNT(*)      OVER (PARTITION BY LOWER(email)) AS  
submissions  
    FROM leads  
)  
SELECT lead_id, company_name, email, created_at, submission_no  
FROM numbered  
WHERE submissions > 1  
ORDER BY email, submission_no;
```

lead_id	company_name	email	created_at	submission_no
2	Bright Kitchens	brightkitchens@example.com	2025-06-12 14:20:00	1
3	Bright Kitchens	brightkitchens@example.com	2025-06-12 15:05:00	2
4	BRIGHT KITCHENS	brightkitchens@example.com	2025-06-12 17:20:00	3
42	Daily Fresh Mart	dailyfreshmart@example.com	2025-11-25 10:05:00	1
43	Daily Fresh Mart	dailyfreshmart@example.com	2025-11-25 10:50:00	2
8	Fortune Foods	fortunefoods@example.com	2025-03-10 17:35:00	1
9	Fortune Foods	fortunefoods@example.com	2025-03-10 17:42:00	2
10	FORTUNE FOODS	fortunefoods@example.com	2025-03-10 20:35:00	3
14	Jasmine Retail	jasmineretail@example.com	2025-11-09 11:20:00	1
15	Jasmine Retail	jasmineretail@example.com	2025-11-09 12:05:00	2
19	Nova Supermart	novasupermart@example.com	2025-01-14 10:20:00	1
20	Nova Supermart	novasupermart@example.com	2025-01-14 10:27:00	2
21	NOVA SUPERMART	novasupermart@example.com	2025-01-14 10:27:00	3
25	Ruby Traders	rubytraders@example.com	2025-09-13 12:20:00	1
27	RUBY TRADERS	rubytraders@example.com	2025-09-13 13:05:00	2
26	Ruby Traders	rubytraders@example.com	2025-09-13 15:20:00	3
31	Vista Resorts	vistaresorts@example.com	2025-04-02 15:35:00	1
32	Vista Resorts	vistaresorts@example.com	2025-04-02 15:42:00	2
36	Zenith Supplies	zenithsupplies@example.com	2025-05-06 17:20:00	1
37	Zenith Supplies	zenithsupplies@example.com	2025-05-06 17:22:00	2
38	ZENITH SUPPLIES	zenithsupplies@example.com	2025-05-06 18:05:00	3

(21 rows)

```
SELECT COUNT(*) AS lead_rows, COUNT(DISTINCT LOWER(email)) AS unique_enquiries
FROM leads;
```

lead_rows	unique_enquiries
43	30

(1 row)

How it works.

- **PARTITION BY LOWER(email)** groups submissions by email address, ignoring capital letters. Matching on `company_name` would fail: "Bright Kitchens" and "BRIGHT KITCHENS" are different text.
- **ORDER BY created_at, lead_id** makes submission 1 the earliest. Look at Nova Supermart: two rows arrived in the same minute. Without `lead_id` as a tie-breaker, which one counts as "first" could change between runs.
- **COUNT(*) OVER (PARTITION BY ...)** adds each email's total number of submissions to every row, so the outer query can show only emails that were duplicated.

To use the clean list in any later query, keep `WHERE submission_no = 1`. You'll do exactly that in the funnel pattern next.

Deciding which row to keep is a business rule. The earliest submission shows when the customer first got in touch. The latest might have the most complete details. Some teams merge the best field from each. Ask the CRM owner before deleting anything, and never delete from the source system as part of an analysis; clean in your query or in a separate cleaned table (Chapter 14).

Where else you'll use it. Duplicate customer records, repeated sensor readings, invoices loaded twice by a pipeline, the current row from a history table.

Pattern 4: Funnel conversion

The question. "Of the enquiries we received, how many did we contact, quote, and win?"

The idea. Count unique leads that reached each stage, in stage order, then compare each stage with the one before it (`LAG`) and with the first stage (`FIRST_VALUE`). This pattern chains Pattern 3 inside it.

```
WITH first_submissions AS (
  SELECT lead_id
  FROM (
    SELECT lead_id,
           ROW_NUMBER() OVER (PARTITION BY LOWER(email) ORDER BY created_at, lead_id) AS
    submission_no
    FROM leads
  ) AS numbered
  WHERE submission_no = 1
),
stage_counts AS (
  SELECT h.stage,
         CASE h.stage WHEN 'New' THEN 1 WHEN 'Contacted' THEN 2
                     WHEN 'Quoted' THEN 3 WHEN 'Won' THEN 4 END AS step,
         COUNT(*) AS leads
  FROM lead_stage_history AS h
  JOIN first_submissions AS f ON h.lead_id = f.lead_id
  WHERE h.stage IN ('New', 'Contacted', 'Quoted', 'Won')
  GROUP BY h.stage
)
SELECT step,
       stage,
       leads,
       ROUND(100.0 * leads / LAG(leads) OVER (ORDER BY step), 1) AS pct_of_previous_step,
       ROUND(100.0 * leads / FIRST_VALUE(leads) OVER (ORDER BY step), 1) AS pct_of_new_leads
FROM stage_counts
ORDER BY step;
```

step	stage	leads	pct_of_previous_step	pct_of_new_leads
1	New	30		100.0
2	Contacted	22	73.3	73.3
3	Quoted	14	63.6	46.7
4	Won	6	42.9	20.0

(4 rows)

How it works.

- `first_submissions` removes duplicates first. Without it, the funnel would start with 43 "new" leads instead of 30, and the win rate would drop from 20% to 14%, blaming the sales team for a website glitch.
- The `CASE` gives each stage a number, because stage *names* don't sort in funnel order alphabetically.
- `LAG(leads)` compares each stage with the previous one; `FIRST_VALUE(leads)` compares it with the top of the funnel. `FIRST_VALUE` works correctly with the default frame, because the first row is always inside it.

Reading it. 8 of 30 enquiries were never even contacted: that's the cheapest leak to fix. Once a quote goes out, 43% become customers, so getting more leads to *the quote stage* is likely worth more than improving the final close.

Where else you'll use it. Website visit → cart → checkout → payment; job applicants → interview → offer → joined; service tickets opened → assigned → resolved.

Pattern 5: Fill in the missing months

The question. "How did Garden Chair sales change month to month?"

The idea. A `GROUP BY` only returns months that **have** data. Garden Chairs sold in March and May, and not at all in April. Watch what `LAG` does with that:

```
WITH furniture AS (  
    SELECT DATE_TRUNC('month', order_date)::date AS month, SUM(net_revenue) AS revenue  
    FROM sales_lines  
    WHERE category = 'Furniture'  
    GROUP BY DATE_TRUNC('month', order_date)  
)  
SELECT month, ROUND(revenue, 0) AS revenue, ROUND(LAG(revenue) OVER (ORDER BY month), 0) AS  
previous_month  
FROM furniture  
ORDER BY month;
```

month	revenue	previous_month
2025-03-01	16388	
2025-05-01	17250	16388

(2 rows)

The query claims May's "previous month" was ₹16,388. It wasn't; April was zero. `LAG` means *previous row*, not *previous month*, and April has no row. The fix is a **date spine**: a complete list of months, left-joined to the data so empty months appear as zero.

```
WITH months AS (  
    SELECT generate_series(DATE '2025-01-01', DATE '2025-12-01', INTERVAL '1 month')::date AS month  
)  
,  
furniture AS (  
    SELECT DATE_TRUNC('month', order_date)::date AS month, SUM(net_revenue) AS revenue  
    FROM sales_lines  
    WHERE category = 'Furniture'  
    GROUP BY DATE_TRUNC('month', order_date)  
)  
,  
filled AS (  
    SELECT m.month,  
           COALESCE(f.revenue, 0) AS revenue,  
           LAG(COALESCE(f.revenue, 0)) OVER (ORDER BY m.month) AS previous_month  
    FROM months AS m  
    LEFT JOIN furniture AS f ON f.month = m.month  
)  
SELECT month, ROUND(revenue, 0) AS revenue, ROUND(previous_month, 0) AS previous_month  
FROM filled  
WHERE month BETWEEN '2025-02-01' AND '2025-06-01'  
ORDER BY month;
```

month	revenue	previous_month
2025-02-01	0	0
2025-03-01	16388	0
2025-04-01	0	16388
2025-05-01	17250	0
2025-06-01	0	17250

(5 rows)

How it works.

- `generate_series(start, end, INTERVAL '1 month')` produces one row per month for the whole year, whether or not anything sold.
- The **left join** keeps every month; `COALESCE` turns missing revenue into zero.
- `LAG` runs on the complete list, so "previous" really is the previous month.
- The `WHERE` filter to February–June is in the **final** step, *after* the window is calculated. Filter in `filled` instead and February would lose its January neighbor.

Dialect note. `generate_series` is PostgreSQL. Snowflake uses `GENERATOR`, BigQuery `GENERATE_DATE_ARRAY`, SQL Server 2022 `GENERATE_SERIES` (for numbers), MySQL a **recursive CTE** (section 13.8), and older systems a small permanent **calendar table**. Most data warehouses have a calendar table for exactly this purpose; ask whether yours does before building one.

Reading it. Furniture is a seasonal, spring-only product for Riverstone. A report that silently skipped the empty months would have shown a steady product.

Where else you'll use it. Any time series with gaps: daily website visits, weekly production, hourly sensor readings. Charts need the gaps too, or the line jumps straight from March to May as if April never happened.

Pattern 6: At-risk customers (breaks in the rhythm)

The question. "Which regular customers have stopped ordering?"

The idea. Use `LAG` to measure each customer's usual gap between orders, then compare the time since their last order with that usual gap. A customer who normally orders every three weeks and hasn't ordered in two months is at risk; a customer who orders twice a year is not, after two months.

```

WITH order_days AS (
    SELECT DISTINCT customer_id, order_date
    FROM sales_lines
),
gaps AS (
    SELECT customer_id,
           order_date,
           order_date - LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS
gap_days
    FROM order_days
),
customer_rhythm AS (
    SELECT customer_id,
           COUNT(*)           AS orders,
           ROUND(AVG(gap_days)) AS usual_gap_days,
           MAX(order_date)     AS last_order
    FROM gaps
    GROUP BY customer_id
    HAVING COUNT(*) >= 4
)
SELECT c.customer_name,
       r.orders,
       r.usual_gap_days,
       r.last_order,
       DATE '2025-12-31' - r.last_order AS days_since_last,
       CASE WHEN DATE '2025-12-31' - r.last_order > 2 * r.usual_gap_days
            THEN 'At risk' ELSE 'On rhythm' END AS status
FROM customer_rhythm AS r
JOIN customers      AS c ON r.customer_id = c.customer_id
ORDER BY (DATE '2025-12-31' - r.last_order) / r.usual_gap_days DESC
LIMIT 8;

```

customer_name	orders	usual_gap_days	last_order	days_since_last	status
Sunrise Caterers	4	37	2025-06-10	204	At risk
Om Sai Provisions	4	39	2025-07-22	162	At risk
Green Leaf Hotels	15	23	2025-11-26	35	On rhythm
Sharma Hardware	16	21	2025-12-05	26	On rhythm
Patel Kitchenware	7	52	2025-11-09	52	On rhythm
Western Logistics	6	23	2025-12-10	21	On rhythm
Northgate Distributors	12	27	2025-12-08	23	On rhythm
Coastal Foods	12	29	2025-12-10	21	On rhythm

(8 rows)

How it works.

- `order_days` uses `DISTINCT` so two orders on the same day count as one visit, rather than creating a misleading zero-day gap.
- `gaps` measures days between visits. The first visit's gap is `NULL`, and `AVG` ignores `NULL`s (section 12.9), so the average covers real gaps only.
- `HAVING COUNT(*) >= 4` limits the report to customers with enough history to *have* a rhythm. Two orders are not a pattern.
- The final `ORDER BY` sorts by how many "usual gaps" have passed since the last order: the most overdue first.

Reading it. Sunrise Caterers and Om Sai Provisions used to order every five to six weeks and have now been silent for over five months. They're probably lost, and someone should find out why. Green Leaf Hotels is worth a friendly call: 35 days is already half again its usual 23-day rhythm.

Where else you'll use it. Subscription renewals, patients overdue for a check-up, machines overdue for maintenance, suppliers whose deliveries have become irregular.

Pattern 7: Cohort retention

The question. "Do customers who start with us keep ordering?"

The idea. Group customers into **cohorts** by when they first ordered (here, the quarter). For each cohort, work out what share of customers ordered again one, two, and three months after their first month. Comparing cohorts shows whether retention is improving.

Think of it like a gym. Everyone who joined in January is one cohort. The gym doesn't just want to know how many members it has; it wants to know how many January joiners still come in February, March, and April, and whether March joiners stick around better.

```
WITH first_orders AS (
    SELECT customer_id, DATE_TRUNC('month', MIN(order_date))::date AS first_month
    FROM sales_lines
    GROUP BY customer_id
),
active_months AS (
    SELECT DISTINCT customer_id, DATE_TRUNC('month', order_date)::date AS month
    FROM sales_lines
),
cohort_activity AS (
    SELECT f.customer_id,
           'Q' || EXTRACT(QUARTER FROM f.first_month) AS cohort,
           (EXTRACT(YEAR FROM a.month) * 12 + EXTRACT(MONTH FROM a.month))
           - (EXTRACT(YEAR FROM f.first_month) * 12 + EXTRACT(MONTH FROM f.first_month)) AS
months_since_first
    FROM first_orders AS f
    JOIN active_months AS a ON f.customer_id = a.customer_id
    WHERE f.first_month < DATE '2025-10-01'
)
SELECT cohort,
       COUNT(DISTINCT customer_id) AS customers,
       ROUND(100.0 * COUNT(DISTINCT CASE WHEN months_since_first = 1 THEN customer_id END) /
COUNT(DISTINCT customer_id), 0) AS month_1_pct,
       ROUND(100.0 * COUNT(DISTINCT CASE WHEN months_since_first = 2 THEN customer_id END) /
COUNT(DISTINCT customer_id), 0) AS month_2_pct,
       ROUND(100.0 * COUNT(DISTINCT CASE WHEN months_since_first = 3 THEN customer_id END) /
COUNT(DISTINCT customer_id), 0) AS month_3_pct
FROM cohort_activity
GROUP BY cohort
ORDER BY cohort;
```

cohort	customers	month_1_pct	month_2_pct	month_3_pct
Q1	11	64	91	82
Q2	4	75	75	75
Q3	6	67	83	67

(3 rows)

How it works.

- `first_orders` finds each customer's first month; `active_months` lists every month each customer ordered.
- `months_since_first` converts both months to a running month number ($\text{year} \times 12 + \text{month}$) and subtracts, so January to March is 2 even across a year boundary.
- `COUNT(DISTINCT CASE WHEN months_since_first = 1 THEN customer_id END)` counts customers active exactly one month after starting. The `CASE` returns NULL for everyone else, and `COUNT` skips NULLs.
- **Q4 is excluded on purpose.** A customer who started in November can't have a "month 3" yet, because the data ends in December. Including them would make Q4 retention look terrible for a reason that has nothing to do with the customers. This is called **right-censoring**, and ignoring it is one of the most common errors in retention reporting.

Reading it. These percentages measure "ordered in that month", not "still a customer". That's why month 2 can be higher than month 1: many B2B customers order every six to eight weeks, skipping a month. Retention is healthy across all three cohorts. With only 4 to 11 customers per cohort, though, a single customer moves a percentage by 9 to 25 points, so treat differences between cohorts as questions, not conclusions (Chapter 22).

Where else you'll use it. App users by sign-up week, subscribers by first plan, employees by joining month (attrition), and marketing campaigns by the month a customer was acquired.

Pattern 8: Streaks (gaps and islands)

The question. "Which customers ordered every single month, and for how long?"

The idea. This is a famous SQL puzzle called **gaps and islands**. Consecutive months with orders form "islands"; missing months are "gaps". The trick: number each customer's active months 1, 2, 3... with `ROW_NUMBER()`. For consecutive months, the month number and the row number go up together, so **their difference stays the same**. A gap breaks it. That difference identifies each island.

Here's the trick on one customer, Patel Kitchenware:

```
WITH months AS (
    SELECT DISTINCT customer_id, DATE_TRUNC('month', order_date)::date AS month
    FROM sales_lines
    WHERE customer_id = 2
)
SELECT month,
       EXTRACT(YEAR FROM month) * 12 + EXTRACT(MONTH FROM month) AS
month_number,
       ROW_NUMBER() OVER (ORDER BY month) AS row_num,
       EXTRACT(YEAR FROM month) * 12 + EXTRACT(MONTH FROM month)
       - ROW_NUMBER() OVER (ORDER BY month) AS island_id
FROM months
ORDER BY month;
```

month	month_number	row_num	island_id
2025-01-01	24301	1	24300
2025-03-01	24303	2	24301
2025-04-01	24304	3	24301
2025-05-01	24305	4	24301
2025-06-01	24306	5	24301
2025-08-01	24308	6	24302
2025-11-01	24311	7	24304

(7 rows)

Patel Kitchenware's order months in 2025



The trick: month_number – row_number stays the same while months are consecutive.

month	month_number	row_number	island_id
Jan	24301	1	24300
Mar	24303	2	24301
Apr	24304	3	24301
May	24305	4	24301
Jun	24306	5	24301
Aug	24308	6	24302
Nov	24311	7	24304

Each consecutive run shares one island_id.
 Group by island_id to get each streak:
Mar-Jun → 4 months in a row (the longest)
 Jan, Aug, Nov → 1 month each
 Real uses: attendance streaks, machine uptime runs.

Figure 13.4 – March to June share island_id 24301: a four-month streak.

March to June all have island_id 24301: one island, four months long. Now apply it to every customer and keep each customer's longest streak:

```
WITH months AS (
  SELECT DISTINCT customer_id, DATE_TRUNC('month', order_date)::date AS month
  FROM sales_lines
),
numbered AS (
  SELECT customer_id,
         month,
         EXTRACT(YEAR FROM month) * 12 + EXTRACT(MONTH FROM month)
         - ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY month) AS island_id
  FROM months
),
streaks AS (
  SELECT customer_id, island_id, MIN(month) AS streak_start, MAX(month) AS streak_end, COUNT(*)
  AS months_in_a_row
  FROM numbered
  GROUP BY customer_id, island_id
),
best AS (
  SELECT s.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY months_in_a_row DESC,
  streak_start) AS rn
  FROM streaks AS s
)
SELECT c.customer_name, b.streak_start, b.streak_end, b.months_in_a_row
FROM best AS b
JOIN customers AS c ON b.customer_id = c.customer_id
WHERE b.rn = 1
ORDER BY b.months_in_a_row DESC, c.customer_name
LIMIT 6;
```

customer_name	streak_start	streak_end	months_in_a_row
Coastal Foods	2025-01-01	2025-12-01	12
Metro Mart	2025-01-01	2025-12-01	12
Sharma Hardware	2025-01-01	2025-12-01	12
Fresh Bowl Kitchens	2025-02-01	2025-12-01	11
Green Leaf Hotels	2025-01-01	2025-11-01	11
Harbour Traders	2025-04-01	2025-12-01	9

(6 rows)

How it works. Four named steps: distinct active months → island IDs → one row per streak (`GROUP BY island_id`) → each customer's longest streak ("number, then keep 1" from section 13.4). The query looks long, but each step does one thing, and you can inspect any of them.

Reading it. Three customers ordered in every month of 2025. Those are Riverstone's most dependable accounts, and good candidates for a yearly supply contract. Harbour Traders has ordered every month since joining in April.

Where else you'll use it. Attendance streaks, consecutive days a machine ran without a fault, consecutive months a store hit its target, login streaks in an app.

Pattern 9: Pivot rows into columns

The question. "Show revenue by category, with one column per quarter."

The idea. One row per category, and conditional aggregation (section 12.9) to spread quarters into columns. PostgreSQL has a neater spelling of `SUM(CASE WHEN ...)`: the `FILTER` clause.

```
SELECT category,
       ROUND(SUM(net_revenue) FILTER (WHERE EXTRACT(QUARTER FROM order_date) = 1), 0) AS q1,
       ROUND(SUM(net_revenue) FILTER (WHERE EXTRACT(QUARTER FROM order_date) = 2), 0) AS q2,
       ROUND(SUM(net_revenue) FILTER (WHERE EXTRACT(QUARTER FROM order_date) = 3), 0) AS q3,
       ROUND(SUM(net_revenue) FILTER (WHERE EXTRACT(QUARTER FROM order_date) = 4), 0) AS q4,
       ROUND(SUM(net_revenue), 0) AS full_year
FROM sales_lines
GROUP BY category
ORDER BY full_year DESC;
```

category	q1	q2	q3	q4	full_year
Storage	503257	342422	451179	1001116	2297974
Kitchen	133888	219336	393660	461146	1208030
Industrial	80780	147560	275450	292040	795830
Furniture	16388	17250			33638

(4 rows)

How it works. `SUM(x) FILTER (WHERE condition)` adds up only the rows that meet the condition, exactly like `SUM(CASE WHEN condition THEN x END)`. Notice Furniture's blank Q3 and Q4: summing no rows gives NULL, not zero. Wrap each column in `COALESCE(..., 0)` if the report will be totalled in Excel.

Dialect note. `FILTER` works in PostgreSQL and SQLite. `SUM(CASE WHEN ... THEN ... END)` works everywhere, including MySQL (section 13.8). SQL Server and Snowflake also have a `PIVOT` operator, but conditional aggregation is easier to read and move between databases.

Reading it. Storage products doubled from Q3 to Q4, the festive season, while Industrial barely grew. Kitchen grew every quarter, which is a very different kind of product: steady growth rather than a seasonal spike.

Pattern 10: Data-quality checks before you trust a report

The question. *"Before I send this report, is the data fit to use?"*

The idea. A set of small checks, each counting problem rows, stacked into one result with `UNION ALL`. Run it before any important report, and make it the first step of any automated one (Chapter 20).

```
SELECT 'Duplicate order IDs' AS check_name, COUNT(*) AS problem_rows
FROM (SELECT order_id FROM orders GROUP BY order_id HAVING COUNT(*) > 1) AS d
UNION ALL
SELECT 'Order lines with no matching order', COUNT(*)
FROM order_items AS oi LEFT JOIN orders AS o ON oi.order_id = o.order_id
WHERE o.order_id IS NULL
UNION ALL
SELECT 'Orders with no lines', COUNT(*)
FROM orders AS o LEFT JOIN order_items AS oi ON o.order_id = oi.order_id
WHERE oi.order_item_id IS NULL
UNION ALL
SELECT 'Non-positive quantities', COUNT(*)
FROM order_items WHERE quantity <= 0
UNION ALL
SELECT 'Orders with no sales rep', COUNT(*)
FROM orders WHERE sales_rep_id IS NULL
UNION ALL
SELECT 'Customers with no orders', COUNT(*)
FROM customers AS c LEFT JOIN orders AS o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL
UNION ALL
SELECT 'Duplicate lead emails (extra rows)', COUNT(*) - COUNT(DISTINCT LOWER(email))
FROM leads;
```

check_name	problem_rows
Duplicate order IDs	0
Order lines with no matching order	0
Orders with no lines	0
Non-positive quantities	0
Orders with no sales rep	11
Customers with no orders	1
Duplicate lead emails (extra rows)	13

(7 rows)

How it works. Each `SELECT` is an independent check that returns one row: a name and a count. `UNION ALL` stacks them. A zero means the check passed. The first four checks guard against broken data; the last three flag business issues that a reader should know about.

Reading it. The structure is sound. But 11 orders have no sales rep, which will quietly understate every rep's numbers, and the lead table is inflated by 13 duplicate rows. Both belong in a footnote on any report that uses these tables, and in a conversation with the people who own the CRM.

Where else you'll use it. Every pipeline you'll build in Part V starts with checks like these. Chapter 47 turns them into automated tests.

13.8 Running this chapter in MySQL

If your company runs MySQL, good news: **every tool in this chapter works there**. MySQL has supported CTEs, recursive CTEs, views, and the full set of window functions (`ROW_NUMBER` , `RANK` , `DENSE_RANK` , `LAG` , `LEAD` , `FIRST_VALUE` , `LAST_VALUE` , running and moving frames, and even the named `WINDOW` clause) since version 8.0. The companion file `ch13_queries_mysql.sql` has every query in this chapter rewritten for MySQL. Each was run against the MySQL versions of both practice databases, and every result matched the PostgreSQL output shown in this chapter.

Setting up

1. Run `riverstone_2025_setup_mysql.sql` from the companion files. Like the Chapter 12 script, it creates its own database (`riverstone_2025`), so there's no separate `CREATE DATABASE` step. Check it: `SELECT COUNT(*) FROM riverstone_2025.orders;` should return `175` .
2. Create the `sales_lines` view in **both** databases, `riverstone` and `riverstone_2025` . The `CREATE VIEW` statement from section 13.2 runs in MySQL unchanged. Run `USE riverstone;` before it the first time and `USE riverstone_2025;` the second time.

What changes

The window functions themselves never change. What changes is the same small set of date and text spellings from section 12.16, plus two things that only appear in this chapter:

In this chapter (PostgreSQL)	In MySQL	Where it appears
<code>DATE_TRUNC('month', order_date)::date</code>	<code>CAST(DATE_FORMAT(order_date, '%Y-%m-01') AS DATE)</code>	every monthly query
<code>order_date - LAG(order_date) OVER (...)</code>	<code>DATEDIFF(order_date, LAG(order_date) OVER (...))</code>	13.5, Patterns 6 and 11
<code>generate_series(...)</code> for a date spine	<code>WITH RECURSIVE</code> (below)	Pattern 5
<code>SUM(x) FILTER (WHERE ...)</code>	<code>SUM(CASE WHEN ... THEN x ELSE 0 END)</code>	Pattern 9
<code>'Q' quarter</code>	<code>CONCAT('Q', quarter)</code>	Pattern 7, exercise 6
a column named <code>rank</code>	<code>rank_no</code> , or quote it as <code>`rank`</code>	13.4

That last row explains a small choice in section 13.4: the ranking columns are called `rank_no` and `dense_rank_no` , not `rank` and `dense_rank` . Since version 8.0, MySQL treats `RANK` and `DENSE_RANK` as **reserved words**, so `... AS rank` is a syntax error there. Other reserved words that analysts trip over as column names include `ROWS` , `GROUPS` , `WINDOW` , and `CHANGE` . Descriptive names such as `rank_in_segment` avoid the problem in every database, and they're clearer anyway.

Month-over-month growth in MySQL

The query from section 13.6, with only the month calculation changed:

```

WITH monthly AS (
    SELECT CAST(DATE_FORMAT(order_date, '%Y-%m-01') AS DATE) AS month,
           SUM(net_revenue) AS revenue
    FROM sales_lines
    GROUP BY CAST(DATE_FORMAT(order_date, '%Y-%m-01') AS DATE)
)
SELECT month,
       ROUND(revenue, 0) AS revenue,
       ROUND(LAG(revenue) OVER w, 0) AS previous_month,
       ROUND(revenue - LAG(revenue) OVER w, 0) AS change_amount,
       ROUND(100.0 * (revenue - LAG(revenue) OVER w) / LAG(revenue) OVER w, 1) AS pct_change
FROM monthly
WINDOW w AS (ORDER BY month)
ORDER BY month;

```

month	revenue	previous_month	change_amount	pct_change
2025-01-01	202640	NULL	NULL	NULL
2025-02-01	253664	202640	51024	25.2
2025-03-01	278008	253664	24344	9.6
2025-04-01	210282	278008	-67726	-24.4
2025-05-01	329359	210282	119077	56.6
2025-06-01	186928	329359	-142431	-43.2
2025-07-01	232692	186928	45764	24.5
2025-08-01	329282	232692	96590	41.5
2025-09-01	558315	329282	229033	69.6
2025-10-01	681071	558315	122756	22.0
2025-11-01	633408	681071	-47663	-7.0
2025-12-01	439824	633408	-193585	-30.6

Every number matches section 13.6. The `WINDOW w AS (...)` clause, `LAG`, and the percentage calculation are written identically in both databases.

A date spine with a recursive CTE

MySQL has no `generate_series`, but it can build the list of months with a **recursive CTE**: a CTE that refers to itself. Here is Pattern 5's Garden Chair query in MySQL:

```

WITH RECURSIVE months AS (
    SELECT DATE '2025-01-01' AS month          -- the first month
    UNION ALL
    SELECT month + INTERVAL 1 MONTH          -- add one month...
    FROM months
    WHERE month < DATE '2025-12-01'          -- ...until December
),
furniture AS (
    SELECT CAST(DATE_FORMAT(order_date, '%Y-%m-01') AS DATE) AS month,
           SUM(net_revenue) AS revenue
    FROM sales_lines
    WHERE category = 'Furniture'
    GROUP BY CAST(DATE_FORMAT(order_date, '%Y-%m-01') AS DATE)
),
filled AS (
    SELECT m.month,
           COALESCE(f.revenue, 0) AS revenue,
           LAG(COALESCE(f.revenue, 0)) OVER (ORDER BY m.month) AS previous_month
    FROM months AS m
    LEFT JOIN furniture AS f ON f.month = m.month
)
SELECT month, ROUND(revenue, 0) AS revenue, ROUND(previous_month, 0) AS previous_month
FROM filled
WHERE month BETWEEN '2025-02-01' AND '2025-06-01'
ORDER BY month;

```

month	revenue	previous_month
2025-02-01	0	0
2025-03-01	16388	0
2025-04-01	0	16388
2025-05-01	17250	0
2025-06-01	0	17250

How the recursive CTE works. Read `months` as a loop in two parts joined by `UNION ALL` :

1. **The starting row** (`SELECT DATE '2025-01-01'`) runs once and produces January.
2. **The recursive part** (`SELECT month + INTERVAL 1 MONTH FROM months ...`) reads the row just produced and adds a month. It keeps running on each new row until its `WHERE` stops being true. After it produces December, `month < '2025-12-01'` is false, and the loop ends.

The rest of the query is unchanged from Pattern 5. `WITH RECURSIVE` is written once, at the start of the `WITH` clause, even though only the first CTE is recursive. `WITH RECURSIVE` works in PostgreSQL too (there the step is written `month + INTERVAL '1 month'`), so a recursive CTE is the date-spine method you can take to either database with the smallest change. Chapter 28 uses recursive CTEs for another classic job: walking an org chart from the managing director down, however many levels deep it goes.

Watch out: MySQL's 1,000-row safety limit. To stop a badly written recursive CTE from running forever, MySQL aborts one that goes past 1,000 steps, with *"Recursive query aborted after 1001 iterations"*. A monthly spine is far below that, but a **daily** spine for three years needs 1,096 rows and fails. For a longer spine, raise the limit for your session first:

```
SET SESSION cte_max_recursion_depth = 5000;
```

Better still, for dates you'll use again and again, ask whether your company has a permanent calendar table, as section 13.7 suggests.

Pivots without FILTER

MySQL doesn't have the `FILTER` clause, so Pattern 9 uses conditional aggregation, the portable form you learned in section 12.9. `QUARTER(order_date)` is MySQL's short spelling of `EXTRACT(QUARTER FROM order_date)` ; both work.

```
SELECT category,
       ROUND(SUM(CASE WHEN QUARTER(order_date) = 1 THEN net_revenue ELSE 0 END), 0) AS q1,
       ROUND(SUM(CASE WHEN QUARTER(order_date) = 2 THEN net_revenue ELSE 0 END), 0) AS q2,
       ROUND(SUM(CASE WHEN QUARTER(order_date) = 3 THEN net_revenue ELSE 0 END), 0) AS q3,
       ROUND(SUM(CASE WHEN QUARTER(order_date) = 4 THEN net_revenue ELSE 0 END), 0) AS q4,
       ROUND(SUM(net_revenue), 0) AS full_year
FROM sales_lines
GROUP BY category
ORDER BY full_year DESC;
```

category	q1	q2	q3	q4	full_year
Storage	503257	342422	451179	1001116	2297974
Kitchen	133888	219336	393660	461146	1208030
Industrial	80780	147560	275450	292040	795830
Furniture	16388	17250	0	0	33638

Notice Furniture's Q3 and Q4 are now **0**, not blank. Adding `ELSE 0` to the `CASE` means every quarter adds up at least one zero, so an empty quarter totals zero instead of `NULL`. That's usually what a report reader wants, and it's a small, deliberate difference from the `FILTER` version in Pattern 9.

The patterns that transfer unchanged

Everything else in section 13.7 runs in MySQL as written, apart from the date spellings in the table above: top N per group, ABC analysis, deduplication with `ROW_NUMBER` and `LOWER(email)` , the funnel with `LAG` and `FIRST_VALUE` , gaps and islands, and the `UNION ALL` data-quality checks.

One behavior to keep in mind from section 12.16: MySQL's default collation ignores capital letters. In Pattern 3, `PARTITION BY LOWER(email)` is essential in PostgreSQL; in MySQL `PARTITION BY email` would already group `Ravi@Example.com` with `ravi@example.com` . Keep the `LOWER()` anyway. It makes the business rule visible to the reader, and the query keeps working when it moves to a warehouse that *does* care about case.

Interview extra point. A classic follow-up to any window-function answer is "How would you do this in MySQL 5.7?", because many older company systems still run versions without window functions. You don't need to write the old workaround from memory. Say what the tool does, then name the fallback: "MySQL before 8.0 has no window functions, so for latest-order-per-customer I'd join each order to a subquery of `MAX(order_date)` per customer, adding `order_id` to break ties. It's slower and harder to read, which is one reason teams upgrade." That shows you understand the problem, not just one database's syntax.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Putting a window function in <code>WHERE</code> or <code>HAVING</code>	Error: <i>window functions are not allowed in WHERE</i>	Calculate it in a CTE, filter in the next step
<code>ROW_NUMBER()</code> with no tie-breaker	"Latest" or "top" rows change between runs	Add a unique column to the window's <code>ORDER BY</code>
Using <code>RANK</code> when the business meant <code>ROW_NUMBER</code> , or the reverse	Four winners in a "top 3", or a tied customer left out	Ask how ties should be handled before writing the query
Forgetting <code>PARTITION BY</code>	<code>LAG</code> compares one customer's order with another customer's; ranks run across all groups	Ask "should this restart for each group?"
Running total with the default frame on tied values	Several rows show the same total, then it jumps	Write <code>ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW</code> and add a tie-breaker
<code>LAST_VALUE</code> with the default frame	Returns the current row's value	Use a full frame, or <code>FIRST_VALUE</code> with reversed order
<code>LAG</code> on data with missing periods	"Previous month" is really two months ago	Build a date spine and left-join to it
Filtering before calculating shares or neighbors	Shares add up to 100% within the filtered rows; first row loses its <code>LAG</code>	Calculate the window first, filter in a later step
Moving average at the start of a series	First months look smoother or lower than they are	Show the number of rows in the frame, or start the series later
Cohort report including cohorts too new to measure	Newest cohort looks like it has terrible retention	Exclude cohorts that can't have reached the period yet
Counting duplicate records as separate events	Funnels, lead counts, and conversion rates are wrong	Deduplicate first with <code>ROW_NUMBER() ... = 1</code>
One giant query with no names	Nobody, including you, can check it	Break it into CTE steps and inspect each one

In the real world: Riverstone's year-end business review

In January 2026, Riverstone's managing director asks for a one-page review of 2025 for the board. Two years ago this took the sales coordinator a week of copying numbers between spreadsheets. With this chapter's patterns, it's an afternoon of work, and next year it's a re-run:

Board question	Pattern	Answer from the queries in this chapter
Did we hit target?	Running total vs target (13.6)	Yes: ₹43.4 lakh, 102.3% of target, after trailing for ten months
When do we sell?	Month-over-month and moving average (13.6), pivot (Pattern 9)	Flat January–August, then a festive surge; Q4 Storage sales more than doubled Q3
How concentrated are we?	ABC analysis (Pattern 2)	11 of 23 customers make 80% of revenue; no single customer above 12%
Who matters most in each segment?	Top N per group (Pattern 1)	Sharma Hardware is a third of Retail
Are we keeping customers?	Cohort retention (Pattern 7), streaks (Pattern 8)	Healthy repeat ordering; three customers ordered every month
Who are we losing?	At-risk customers (Pattern 6)	Sunrise Caterers and Om Sai Provisions have gone silent
Is the pipeline healthy?	Deduplication and funnel (Patterns 3 and 4)	30 real enquiries, 20% won; 8 were never contacted
Can we trust these numbers?	Data-quality checks (Pattern 10)	Structurally sound; 11 orders missing a sales rep, 13 duplicate leads

Notice that the table's middle column is a list of *patterns*, not queries. That's how experienced analysts think: they don't start from a blank editor, they recognize the shape of a question. Save each query from this chapter with a comment explaining what it answers, and you have the start of a personal SQL library that will serve you for years.

Tools

- **PostgreSQL** and **DBeaver**, as in Chapter 12. Every feature in this chapter is standard SQL except `generate_series`, `FILTER`, and the `::date` shorthand, which are flagged where they appear.
- **MySQL 8.0 or later** (optional). Every window function, CTE, and view in this chapter works there; section 13.8 lists the few spellings that change. Companion files: `riverstone_2025_setup_mysql.sql` and `ch13_queries_mysql.sql`.
- **A SQL formatter**. Long CTE queries are only readable if they're consistently laid out. DBeaver's `Format SQL` command is a good start; Chapter 32 introduces automatic formatting ("linting") for teams.
- **A snippets file or repository**. Keep your pattern library in a Git repository (Chapter 26), one file per pattern, with a comment at the top saying what question it answers.
- **Later**: Power BI's DAX language (Chapter 16) has its own versions of running totals, ranking, and time comparisons. Knowing the SQL versions makes the DAX versions much easier to understand.

The project: a monthly business review pack

Goal: a set of saved, commented queries that produce Riverstone's monthly business review, ready to re-run every month.

Data: the one-year database, or your own company's data, with permission. The patterns work on any orders data.

Steps:

1. **Agree the definitions.** Write down, in plain English, what counts as revenue (which statuses?), what counts as an active customer, and what "at risk" means. Put the revenue rule into a view like `sales_lines`.
2. **Run the health check first.** Adapt Pattern 10 to your data. Fix or footnote anything it finds.
3. **Build the headline.** Monthly revenue with month-over-month change, year-to-date against target, and a 3-month moving average (section 13.6). Use a date spine so months with no sales still appear.
4. **Add the breakdowns.** Top N customers per segment (Pattern 1), ABC analysis (Pattern 2), and a category-by-quarter pivot (Pattern 9).
5. **Add the early warnings.** At-risk customers (Pattern 6) and, if you have lead data, a deduplicated funnel (Patterns 3 and 4).
6. **Check every number.** Each breakdown should reconcile to the headline total. Hand-check one `LAG` and one running total.
7. **Write the story.** A one-page memo: three findings, each with a number, what it means, and a suggested action (Chapter 24).
8. **Save it properly.** One file per query, each with a header comment: the question, the business rules, the database, and the date last checked.

Stretch goals:

- Turn the headline query into a single CTE query that outputs all the headline numbers in one row, ready to paste into an email (Chapter 20 automates exactly this).
- Add same-month-last-year comparisons using `LAG(revenue, 12)` on a two-year dataset.
- Build a cohort table with one column per month for months 1 to 6, and highlight any cohort whose month-3 figure drops below 50%.

You've got it when...

- ☐ I can rewrite a nested-subquery query as a CTE, and explain each step in one sentence.
- ☐ I know when a view is a better choice than a CTE, and what business rule my view contains.
- ☐ I can explain the difference between `GROUP BY` and a window function using a real example.

- ☐ I choose between `ROW_NUMBER` , `RANK` , and `DENSE_RANK` deliberately, and always add a tie-breaker to `ROW_NUMBER` .
- ☐ I can write "top N per group" and "latest record per group" without looking anything up.
- ☐ I can calculate month-over-month change, a running total, and a moving average, and I write frames explicitly.
- ☐ I know why `LAG` needs a date spine when periods are missing.
- ☐ I can recognize which of the ten patterns a new business question belongs to.
- ☐ I've built a monthly review pack that reconciles to a known total.

When a manager asks a question with *compared with*, *top*, *running*, *previous*, or *per group* in it, and you already know the shape of the query before you open the editor, this chapter has done its job.

Recap

- A **CTE** (`WITH name AS (...)`) names a step of a query. Chain steps with commas; later steps can use earlier ones. Debug by selecting from any step.
 - A **view** saves a query permanently under a name. Use it to define a business rule, like revenue, once for everyone.
 - A **window function** (`function OVER (...)`) calculates across related rows **without collapsing them**. `PARTITION BY` restarts it for each group; `ORDER BY` sets the order; a **frame** chooses which rows count.
 - Windows run after `WHERE` , `GROUP BY` , and `HAVING` , so filter on them in a later CTE step (or with `QUALIFY` where supported).
 - **ROW_NUMBER** numbers rows uniquely (add a tie-breaker); **RANK** shares ranks and skips; **DENSE_RANK** shares ranks without gaps.
 - **LAG / LEAD** read the previous/next row: month-over-month change, days between orders.
 - **Running totals** use `SUM() OVER (ORDER BY ...)` ; **moving averages** use `AVG() OVER (... ROWS BETWEEN n PRECEDING AND CURRENT ROW)` . Write frames explicitly: the default `RANGE` frame groups ties.
 - The **pattern library**: top N per group, Pareto/ABC, deduplication, funnels, date spines, at-risk customers, cohort retention, gaps and islands, pivots, and data-quality checks.
 - Everything here works in **MySQL 8.0 and later**: change the date spellings, replace `generate_series` with `WITH RECURSIVE` and `FILTER` with `SUM(CASE ...)` , and avoid reserved words such as `rank` as column names.
 - Great analysis is recognizing the pattern, respecting the grain, handling ties and gaps, and reconciling to a number you trust.
-

Practice exercises

Exercise 1 uses the **mini database**; all others use the **one-year database**. Both need the `sales_lines` view. Predict the shape of each result before running it.

Warm-up

1. (Mini database.) Using a CTE, list customers whose total units bought (non-cancelled orders) are more than 100, most units first.
2. Show each product category's revenue and its percentage of total revenue, using a window function rather than a subquery.
3. Number Metro Mart's orders (customer 5) in date order, 1st, 2nd, 3rd..., and show the first five.

Core

4. For each category, show the top two products by revenue. Use `RANK()`.
5. Find each sales rep's best month: the month with their highest revenue, and that revenue.
6. Show quarterly revenue for each customer segment, with the previous quarter's revenue and the percentage change.
7. For customers with at least two orders, show the first order date, the second order date, and the days between them. Show the five longest waits.
8. Show the number of orders per month and a 3-month moving average of that number.

Stretch

9. Run an ABC analysis on **products** instead of customers.
10. For each lead source, after removing duplicate leads, show the number of unique leads, the number won, and the win rate.
11. For customers with at least three orders, find each customer's longest gap between consecutive orders, with the dates on either side of it. Show the five longest.
12. Flag months where revenue was more than 20% below the average of the **previous three months** (not including the month itself).

Think about it (no SQL needed)

13. A report shows "each customer's latest order" using `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC)`. The finance team says a customer's latest order changed between Monday's and Tuesday's report, though no new orders were placed. What's the likely cause, and the fix?
14. A manager looks at a running total of orders and asks why it "jumps by three" on 4 May instead of going up one order at a time. Explain it to her in two sentences without using the words `RANGE` or `frame`.
15. In exercise 10, cold calling shows a 66.7% win rate and the website shows 7.1%. The sales head wants to move the whole marketing budget into cold calling. What would you tell him?

MySQL track (optional)

16. (MySQL, one-year database.) Build a **daily** date spine for December 2025 with a recursive CTE. Use it to show how many days the month had, how many of those days had no orders, and the month's total revenue. Check the total against section 13.6.

Key terms

common table expression (CTE) · WITH · view · temporary table · window function · OVER · PARTITION BY · window ORDER BY · frame · ROWS vs RANGE · UNBOUNDED PRECEDING · CURRENT ROW · named window (WINDOW) · QUALIFY · ROW_NUMBER · RANK · DENSE_RANK · tie-breaker · LAG · LEAD · FIRST_VALUE · LAST_VALUE · running total · year-to-date (YTD) · moving average · month-over-month · date spine / calendar table · generate_series · recursive CTE · cte_max_recursion_depth · reserved word · top N per group · Pareto principle · ABC analysis · deduplication · funnel · conversion rate · cohort · retention · right-censoring · gaps and islands · pivot · FILTER · data-quality check

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 14, Data Cleaning & Preparation**, goes deeper into duplicates, missing values, and messy text, the problems Patterns 3 and 10 only flag.
 - **Chapter 16, Business Intelligence with Power BI**, shows the same ideas (running totals, rankings, time comparisons) in DAX, and how to turn these queries into interactive dashboards.
 - **Chapter 20, Automating Reports & Delivering Insights**, schedules your review pack and emails the headline numbers every month.
 - **Chapter 28, Advanced SQL, Performance & Data Modeling**, covers recursive CTEs, advanced frames, indexes, and query plans, for when these queries meet millions of rows.
 - **Interview preparation:** the SQL Question Bank (Chapter 71) includes the classic window-function rounds: second-highest salary, top N per group, running totals, consecutive days, and deduplication, with graded model answers.
-

Answers to practice exercises

(In the finished book these move to Appendix G. Every query below was run, and the outputs are real.)

1. (Mini database.)

```

WITH customer_units AS (
    SELECT c.customer_name, SUM(s.quantity) AS units
    FROM sales_lines AS s
    JOIN customers AS c ON s.customer_id = c.customer_id
    GROUP BY c.customer_name
)
SELECT customer_name, units
FROM customer_units
WHERE units > 100
ORDER BY units DESC;

```

customer_name	units
Metro Mart	192
Sharma Hardware	113

(2 rows)

2. A window function can wrap an aggregate: the inner `SUM` belongs to `GROUP BY`, the outer `SUM ... OVER ()` adds up the category totals.

```

SELECT category,
       ROUND(SUM(net_revenue), 0) AS revenue,
       ROUND(100.0 * SUM(net_revenue) / SUM(SUM(net_revenue)) OVER (), 1) AS pct_of_total
FROM sales_lines
GROUP BY category
ORDER BY revenue DESC;

```

category	revenue	pct_of_total
Storage	2297974	53.0
Kitchen	1208030	27.9
Industrial	795830	18.4
Furniture	33638	0.8

(4 rows)

3. `sales_lines` has one row per order line, so take distinct orders first:

```

WITH metro_orders AS (
    SELECT DISTINCT order_id, order_date
    FROM sales_lines
    WHERE customer_id = 5
)
SELECT ROW_NUMBER() OVER (ORDER BY order_date, order_id) AS order_number,
       order_id,
       order_date
FROM metro_orders
ORDER BY order_number
LIMIT 5;

```

order_number	order_id	order_date
1	10003	2025-01-12
2	10004	2025-01-13
3	10017	2025-02-25
4	10021	2025-03-18
5	10030	2025-04-08

(5 rows)

Without `DISTINCT`, an order with three lines would get three numbers.

4.

```
WITH product_revenue AS (  
    SELECT p.category, p.product_name, SUM(s.net_revenue) AS revenue  
    FROM sales_lines AS s  
    JOIN products AS p ON s.product_id = p.product_id  
    GROUP BY p.category, p.product_name  
) ,  
ranked AS (  
    SELECT category, product_name, revenue,  
           RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rank_in_category  
    FROM product_revenue  
)  
SELECT category, rank_in_category, product_name, ROUND(revenue, 0) AS revenue  
FROM ranked  
WHERE rank_in_category <= 2  
ORDER BY category, rank_in_category;
```

category	rank_in_category	product_name	revenue
Furniture	1	Garden Chair	33638
Industrial	1	Industrial Crate	795830
Kitchen	1	Lunch Box Set	546630
Kitchen	2	Food Container Set	523745
Storage	1	Storage Box 25L	908213
Storage	2	Storage Box 10L	860946

(6 rows)

Furniture and Industrial have only one product each, so they show one row. "Top two" can return fewer rows than you expect; say so in the report.

5.

```
WITH rep_months AS (  
    SELECT e.employee_name,  
           DATE_TRUNC('month', s.order_date)::date AS month,  
           SUM(s.net_revenue) AS revenue  
    FROM sales_lines AS s  
    JOIN employees AS e ON s.sales_rep_id = e.employee_id  
    GROUP BY e.employee_name, DATE_TRUNC('month', s.order_date)  
) ,  
ranked AS (  
    SELECT employee_name, month, revenue,  
           ROW_NUMBER() OVER (PARTITION BY employee_name ORDER BY revenue DESC, month) AS rn  
    FROM rep_months  
)  
SELECT employee_name, month AS best_month, ROUND(revenue, 0) AS revenue  
FROM ranked  
WHERE rn = 1  
ORDER BY revenue DESC;
```

employee_name	best_month	revenue
Rahul Mehta	2025-10-01	306586
Farah Khan	2025-09-01	280521
Neha Kulkarni	2025-10-01	159955

(3 rows)

6.

```
WITH quarterly AS (  
    SELECT c.segment,  
           EXTRACT(QUARTER FROM s.order_date) AS quarter,  
           SUM(s.net_revenue) AS revenue  
    FROM sales_lines AS s  
    JOIN customers AS c ON s.customer_id = c.customer_id  
    GROUP BY c.segment, EXTRACT(QUARTER FROM s.order_date)  
)  
SELECT segment,  
       'Q' || quarter AS quarter,  
       ROUND(revenue, 0) AS revenue,  
       ROUND(LAG(revenue) OVER w, 0) AS previous_quarter,  
       ROUND(100.0 * (revenue - LAG(revenue) OVER w) / LAG(revenue) OVER w, 1) AS pct_change  
FROM quarterly  
WINDOW w AS (PARTITION BY segment ORDER BY quarter)  
ORDER BY segment, quarter;
```

segment	quarter	revenue	previous_quarter	pct_change
Hospitality	Q1	149040		
Hospitality	Q2	252170	149040	69.2
Hospitality	Q3	307278	252170	21.9
Hospitality	Q4	435551	307278	41.7
Retail	Q1	394328		
Retail	Q2	198474	394328	-49.7
Retail	Q3	333973	198474	68.3
Retail	Q4	562000	333973	68.3
Wholesale	Q1	190944		
Wholesale	Q2	275925	190944	44.5
Wholesale	Q3	479039	275925	73.6
Wholesale	Q4	756751	479039	58.0

(12 rows)

`PARTITION BY segment` is essential: without it, Retail's Q1 would be compared with Hospitality's Q4.

7.

```

WITH customer_orders AS (
    SELECT DISTINCT s.customer_id, s.order_id, s.order_date
    FROM sales_lines AS s
),
with_next AS (
    SELECT customer_id,
           order_date,
           LEAD(order_date) OVER (PARTITION BY customer_id ORDER BY order_date, order_id) AS
next_order_date,
           ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date, order_id) AS
order_number
    FROM customer_orders
)
SELECT c.customer_name,
       w.order_date AS first_order,
       w.next_order_date AS second_order,
       w.next_order_date - w.order_date AS days_to_second_order
FROM with_next AS w
JOIN customers AS c ON w.customer_id = c.customer_id
WHERE w.order_number = 1
      AND w.next_order_date IS NOT NULL
ORDER BY days_to_second_order DESC
LIMIT 5;

```

customer_name	first_order	second_order	days_to_second_order
Lakeview Resorts	2025-05-22	2025-09-18	119
Patel Kitchenware	2025-01-02	2025-03-28	85
Royal Banquets	2025-09-09	2025-11-23	75
Festive Gifts Co	2025-09-15	2025-11-19	65
Blue Bay Cafe	2025-03-21	2025-05-21	61

(5 rows)

The time to a second order is one of the best early signs of whether a new customer will stick. Lakeview Resorts, a seasonal business, waited four months.

8.

```

WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date)::date AS month,
           COUNT(DISTINCT order_id) AS orders
    FROM sales_lines
    GROUP BY DATE_TRUNC('month', order_date)
)
SELECT month,
       orders,
       ROUND(AVG(orders) OVER (ORDER BY month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 1) AS
moving_avg_3m
FROM monthly
ORDER BY month;

```

month	orders	moving_avg_3m
2025-01-01	8	8.0
2025-02-01	10	9.0
2025-03-01	11	9.7
2025-04-01	12	11.0
2025-05-01	15	12.7
2025-06-01	13	13.3
2025-07-01	13	13.7
2025-08-01	14	13.3
2025-09-01	18	15.0
2025-10-01	20	17.3
2025-11-01	21	19.7
2025-12-01	18	19.7

(12 rows)

Compare this with revenue in section 13.6. In June, orders slipped only from 15 to 13, but revenue fell 43%. The average order shrank from about ₹22,000 to about ₹14,400, so the June dip came mostly from **smaller** orders, not fewer ones. That points to a different cause, and a different fix, than losing customers.

9.

```
WITH product_revenue AS (
    SELECT p.product_name, SUM(s.net_revenue) AS revenue
    FROM sales_lines AS s
    JOIN products AS p ON s.product_id = p.product_id
    GROUP BY p.product_name
),
cumulative AS (
    SELECT product_name,
           revenue,
           100.0 * revenue / SUM(revenue) OVER () AS pct_of_total,
           100.0 * SUM(revenue) OVER (ORDER BY revenue DESC, product_name
                                     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
           / SUM(revenue) OVER () AS cumulative_pct
    FROM product_revenue
)
SELECT product_name,
       ROUND(revenue, 0) AS revenue,
       ROUND(cumulative_pct, 1) AS cumulative_pct,
       CASE WHEN cumulative_pct - pct_of_total < 80 THEN 'A'
            WHEN cumulative_pct - pct_of_total < 95 THEN 'B'
            ELSE 'C' END AS abc_class
FROM cumulative
ORDER BY revenue DESC;
```

product_name	revenue	cumulative_pct	abc_class
Storage Box 25L	908213	20.9	A
Storage Box 10L	860946	40.8	A
Industrial Crate	795830	59.2	A
Lunch Box Set	546630	71.8	A
Stackable Bin	528815	84.0	A
Food Container Set	523745	96.0	B
Water Bottle 1L	137655	99.2	C
Garden Chair	33638	100.0	C

(8 rows)

10.

```
WITH first_submissions AS (  
  SELECT lead_id, source  
    FROM (  
      SELECT lead_id, source,  
             ROW_NUMBER() OVER (PARTITION BY LOWER(email) ORDER BY created_at, lead_id) AS  
submission_no  
      FROM leads  
    ) AS numbered  
   WHERE submission_no = 1  
)  
SELECT f.source,  
       COUNT(*)                AS unique_leads,  
       COUNT(w.lead_id)        AS won,  
       ROUND(100.0 * COUNT(w.lead_id) / COUNT(*), 1) AS win_rate_pct  
FROM first_submissions AS f  
LEFT JOIN lead_stage_history AS w  
  ON w.lead_id = f.lead_id  
 AND w.stage = 'Won'  
GROUP BY f.source  
ORDER BY win_rate_pct DESC, unique_leads DESC;
```

source	unique_leads	won	win_rate_pct
Cold call	3	2	66.7
Referral	5	2	40.0
IndiaMART listing	3	1	33.3
Website	14	1	7.1
Trade fair	5	0	0.0

(5 rows)

The `stage = 'Won'` condition is in the `ON` clause so leads that weren't won stay in the count (section 12.10). See exercise 15 before acting on this result.

11.

```

WITH customer_orders AS (
    SELECT DISTINCT customer_id, order_id, order_date
    FROM sales_lines
),
gaps AS (
    SELECT customer_id,
        LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date, order_id) AS
gap_start,
        order_date AS gap_end,
        COUNT(*) OVER (PARTITION BY customer_id) AS
total_orders
FROM customer_orders
),
ranked AS (
    SELECT customer_id, gap_start, gap_end, gap_end - gap_start AS gap_days,
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY gap_end - gap_start DESC,
gap_start) AS rn
FROM gaps
WHERE gap_start IS NOT NULL
    AND total_orders >= 3
)
SELECT c.customer_name, r.gap_start, r.gap_end, r.gap_days
FROM ranked AS r
JOIN customers AS c ON r.customer_id = c.customer_id
WHERE r.rn = 1
ORDER BY r.gap_days DESC, c.customer_name
LIMIT 5;

```

customer_name	gap_start	gap_end	gap_days
Lakeview Resorts	2025-05-22	2025-09-18	119
Blue Bay Cafe	2025-07-15	2025-11-09	117
Patel Kitchenware	2025-08-14	2025-11-09	87
Northgate Distributors	2025-06-13	2025-08-27	75
Royal Banquets	2025-09-09	2025-11-23	75

(5 rows)

Notice the pattern stacked three deep: `LAG` finds gaps, `COUNT(*) OVER` filters to customers with enough orders, and `ROW_NUMBER` keeps the longest gap per customer. The `total_orders` window is calculated in the `gaps` step and filtered in the `ranked` step, never in the same step.

12. The frame `ROWS BETWEEN 3 PRECEDING AND 1 PRECEDING` averages the three months *before* the current one, excluding it:

```

WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date)::date AS month, SUM(net_revenue) AS revenue
    FROM sales_lines
    GROUP BY DATE_TRUNC('month', order_date)
),
smoothed AS (
    SELECT month,
           revenue,
           AVG(revenue) OVER (ORDER BY month ROWS BETWEEN 3 PRECEDING AND 1 PRECEDING) AS
avg_prev_3m
    FROM monthly
)
SELECT month,
       ROUND(revenue, 0) AS revenue,
       ROUND(avg_prev_3m, 0) AS avg_previous_3_months,
       ROUND(100.0 * (revenue - avg_prev_3m) / avg_prev_3m, 1) AS pct_vs_recent_average
FROM smoothed
WHERE revenue < 0.8 * avg_prev_3m
ORDER BY month;

```

month	revenue	avg_previous_3_months	pct_vs_recent_average
2025-06-01	186928	272549	-31.4
2025-12-01	439824	624265	-29.5

(2 rows)

Excluding the current month matters: a bad month shouldn't drag down the average it's being compared with. January has no previous months, so its average is NULL and the comparison is unknown, so it's correctly left out. This is a simple version of the **anomaly alert** you'll automate in Chapter 20.

13. The customer probably has two orders on the same date. With only `order_date` in the window's `ORDER BY`, the database is free to number the tied orders either way, and it can choose differently on different days. The fix is a unique tie-breaker: `ORDER BY order_date DESC, order_id DESC`. Every `ROW_NUMBER()` in a report should end its `ORDER BY` with something unique.

14. "The running total counts each **day** as one step, so the three orders placed on 4 May are all added at once. If you want to see it grow order by order, I can change it to count each order separately."

15. Cold calling's 66.7% is 2 wins out of 3 leads. With samples that small, one more lost lead would drop it to 50%, and one more won website lead would double the website's rate. Before moving any budget: collect more data (a longer period or all past years), look at the **number** of wins and their value rather than only the rate, and consider cost per lead, since cold calls cost salespeople's time. A sensible response is "cold calling looks promising; let's test it properly for a quarter" rather than "move the whole budget". Chapter 22 explains why small samples mislead, and Chapter 30 shows how to run a proper test.

16. (MySQL.) A recursive CTE produces the 31 days; a left join keeps the days with no sales:

```

WITH RECURSIVE days AS (
  SELECT DATE '2025-12-01' AS day
  UNION ALL
  SELECT day + INTERVAL 1 DAY
  FROM days
  WHERE day < DATE '2025-12-31'
),
daily_sales AS (
  SELECT order_date, SUM(net_revenue) AS revenue
  FROM sales_lines
  WHERE order_date >= '2025-12-01' AND order_date < '2026-01-01'
  GROUP BY order_date
)
SELECT COUNT(*) AS days_in_month,
       SUM(CASE WHEN s.order_date IS NULL THEN 1 ELSE 0 END) AS days_without_orders,
       ROUND(SUM(COALESCE(s.revenue, 0)), 0) AS month_revenue
FROM days AS d
LEFT JOIN daily_sales AS s ON s.order_date = d.day;

```

```

+-----+-----+-----+
| days_in_month | days_without_orders | month_revenue |
+-----+-----+-----+
|          31 |          17 |      439824 |
+-----+-----+-----+

```

The revenue matches December in section 13.6 (₹439,824). ✓ Orders came in on only 14 of December's 31 days, which is normal for a B2B supplier whose customers order every few weeks, and a reason daily charts for this kind of business need a date spine: without one, the 17 empty days would simply vanish. `SUM(CASE WHEN s.order_date IS NULL THEN 1 ELSE 0 END)` counts the unmatched days. A 31-row spine is well within MySQL's 1,000-step recursion limit, so no setting needs changing.